System Analysis and Design

Kiarash Sanei

July 2026

Contents

I Foundations **2**

1 Course Overview & Tools **2**

2 System Analysis & Design Fundamentals **2**
2.1 Why Projects Fail . . . 2
2.2 System & Information-System Concepts . . . 2
2.3 Enterprise Applications . . . 2
2.4 Stakeholders & Development-Process Roles . . . 3
2.5 System Analyst Skills . . . 3

3 The Software Development Process **3**
3.1 Methodology Fundamentals . . . 3
3.2 Capability Maturity Model (CMM) . . . 3
3.3 System Life Cycle & High-Level Activities . . . 3
3.4 Structured Design Methods . . . 4
3.5 Rapid Application Development (RAD) . . . 4
3.6 Agile Methods . . . 5
3.7 Methodology Selection (summary table) . . . 5
3.8 Key Deliverables & Cross-Life-Cycle Activities . . . 5

4 System Analysis **5**
4.1 What System Analysis Does . . . 5
4.2 Model-Driven Analysis . . . 5
4.3 Requirements Analysis . . . 5
4.4 Requirements Discovery Methods . . . 5
4.5 Problem Analysis . . . 6
4.6 Feasibility Analysis . . . 6
4.7 Economic Evaluation Techniques . . . 6
4.8 Comparing & Choosing Alternatives . . . 6

4.9 System Proposal 6
4.10 Creeping Commitment & Risk Reduction 6

5 Planning & Project Management **7**
5.1 Why Plan? 7
5.2 Project Management Tools 7
5.3 Effort Estimation 7
5.4 Work Plan & WBS 7
5.5 Scope Management 7
5.6 Staffing 7
5.7 Motivation & Conflict 7
5.8 Environment, Infrastructure & Documentation 8
5.9 Release Planning 8
5.10 Project-Management Software 8

II Agile & Scrum **8**

6 Agile Fundamentals **8**
6.1 History 8
6.2 The Agile Manifesto 8
6.3 The Twelve Principles (condensed) 9
6.4 Benefits, Challenges, and Agile vs. Traditional 9
6.5 Agile Metrics 9

7 Scrum **9**
7.1 Background 9
7.2 Roles 9
7.3 Artifacts 10
7.4 States: Definition of Ready vs. Definition of Done 10
7.5 Events 10
7.6 Social Objects 10
7.7 Scrum Flow 10
7.8 Benefits 10

8 Requirements in Scrum (User Stories) **11**
8.1 Requirements: Sequential vs. Scrum 11
8.2 User Stories and the 3 Cs 11
8.3 Levels of Detail 11
8.4 INVEST — Evaluating Stories 11
8.5 Non-Functional Requirements & Knowledge-Acquisition Stories . . 12

8.6 Gathering & Mapping Stories 12
8.7 PBI Types 12
8.8 Grooming and the DEEP Model 12
8.9 DoR & Release-Flow Management 12

9 Sprints in Scrum **12**
9.1 Estimation, Units & Velocity 12
9.2 Planning Poker 13
9.3 Velocity — Use & Misuse 13
9.4 Sprint Planning 13
9.5 Determining Capacity 13
9.6 Sprint Execution & Flow 13
9.7 Burndown & Burnup Charts 13
9.8 Sprint Review 14
9.9 Sprint Retrospective 14

10 eXtreme Programming (XP) **14**
10.1 Overview, Values, Principles 14
10.2 The XP Process (phases) 14
10.3 The 12 XP Practices 15
10.4 System Metaphor & Team 15

III Modeling **15**

11 Requirements & Process Modeling **15**
11.1 Requirements Discovery 15
11.2 Consequences of Flawed Requirements 15
11.3 Requirements Documents & Management 15
11.4 Fact-Finding Methods 15
11.5 Use-Case Modeling 15
11.6 Use-Case Symbols, Actors & Relationships 16
11.7 Process of Requirements Use-Case Modeling 16
11.8 Modeling Categories 17

12 UML Behavioral Diagrams **17**
12.1 What UML Is 17
12.2 UML in Analysis, Design & Documentation 17
12.3 Use Case Diagram 17
12.4 State Machine Diagram 17
12.5 Activity Diagram 17

12.6 Sequence Diagram 18
12.7 Other Behavioral Diagrams & BPMN 18

13 UML Structural Diagrams **18**
13.1 Class Diagrams 18
13.2 Class Diagram Relationships 19
13.3 Component Diagrams 19
13.4 Composite Structure Diagrams 19
13.5 Deployment Diagrams 20

IV Software Design **20**

14 Introduction to System Design **20**
14.1 What System Design Is 20
14.2 From Analysis Models to Design Models 20
14.3 Design Strategy Decision 20
14.4 Software Architecture 20
14.5 Architecture Characteristics 21
14.6 Trade-offs, Technical Debt, and Continuous Improvement 21
14.7 Architecture vs. Design 21
14.8 Subsystems, Interfaces, and Packages 22
14.9 The Design: Classes, Units, Modules, UI 22

15 Architectural Patterns **22**
15.1 MVC (Model-View-Controller) 22
15.2 MVVM (Model-View-ViewModel) 23
15.3 Layered (n-tier) Architecture 23
15.4 Clean Architecture 23
15.5 Microkernel (Plugin) Architecture 23
15.6 Microservices 23

16 Design Patterns (Gang of Four) **24**
16.1 Creational Patterns 24
16.2 Structural Patterns 25
16.3 Behavioral Patterns 27

17 System Design: An Incremental Approach **29**
17.1 Client, Server, and DNS 29
17.2 Databases 29
17.3 Scaling 29

17.4 Caching and CDN . . . 29
17.5 Stateful vs. Stateless Architecture . . . 30
17.6 GeoDNS and Message Queues . . . 30
17.7 Observability and Supporting Infrastructure . . . 30

18 UI/UX Design **30**
18.1 UI vs. UX . . . 30
18.2 What Makes Good UX? . . . 30
18.3 The UX Design Process . . . 31
18.4 Laws of UX . . . 31
18.5 Tools, Systems, and Collaboration . . . 31

V Engineering Practices *(Course Extras)* **32**

19 Introduction to DevOps **32**
19.1 What is DevOps? . . . 32
19.2 The Problems Before DevOps . . . 32
19.3 How DevOps Works . . . 32
19.4 CI/CD . . . 32
19.5 DevOps, Agile, and System Design . . . 32
19.6 The CI/CD Pipeline . . . 33
19.7 Environments, Tools, and Culture . . . 33

20 Docker **33**
20.1 Why Containers . . . 33
20.2 Ecosystem & Concepts . . . 33
20.3 Dockerfile Details . . . 34
20.4 Compose, Volumes, Networks . . . 34

21 DevOps & CI/CD Tools **35**
21.1 Pipelines . . . 35
21.2 The Tools . . . 35
21.3 Real-World Flow . . . 35

22 Microservices in Practice **36**
22.1 From Monolith to Services . . . 36
22.2 Designing Services with DDD . . . 36
22.3 Communication . . . 36
22.4 Building, Releasing, Testing, Operating . . . 36

23 More Software Development Methodologies **37**

24 Software Documentation **37**

25 Software Testing **38**
25.1 Methods & Levels 38
25.2 Coverage, Edge Cases, Critical Paths 38
25.3 Mocking & Test Design 38
25.4 Higher-Level & Specialised Testing 39

VI Epilogue **39**

26 The Epilogue: Looking Back & Forward **39**
26.1 What This Course Covered 39
26.2 Career Paths 39
26.3 AI and the Software Engineer 39
26.4 Soft Skills and Mindset 40
26.5 Practical Next Steps 40

VII Cheat Sheet **40**

27 Methodology Selection "Pick Your Poison" **40**

28 Roles, Requirements & Quick Mnemonics **40**

29 Feasibility & Economics **41**

30 UML & Modeling **41**

31 Pattern Catalogue **41**

32 DevOps, Docker & Testing Quick Reference **42**

33 Microservices & Architecture Decisions **42**

34 System Design Quick Reference **42**

35 UI/UX Laws Quick Reference **43**

36 DevOps Quick Reference **43**

I Foundations

1 Course Overview & Tools

- **The "area computer" analogy.** To do one job well (compute areas) you must master a toolkit (integration): models, processes, and patterns. This course is the toolkit for building information systems.
- **What the course covers:** general practices (project management, feasibility study), methodologies (Scrum, continuous delivery), modeling (diagrams and modeling languages), analysis practices (requirements, process, and data analysis), and design practices (software architecture, design patterns, data model, UI/UX).
- **Grading:** Exercises 5 pt, Project $5 + 1$ pt, Midterm 5 pt, Final 5 pt.

2 System Analysis & Design Fundamentals

2.1 Why Projects Fail

- Between 2011–2015 only about 29% of software projects were fully successful; roughly half were "challenged"; failure rates hovered between 17% and 22%.
- Common failure causes: unrealistic goals, inaccurate resource estimates, and poorly defined requirements.
- Top three success factors: *user involvement*, *executive support*, and *clear requirements*. Lack of user input and incomplete requirements are the leading causes of challenged projects.
- Major software failures are often compared to avoidable airplane crashes caused by inexperience and poor maintenance. Success needs a balance of techniques, methodologies, and tools.

2.2 System & Information-System Concepts

- A **system** is a group of interrelated components working together to achieve a specific result.
- **Information system types:** Transaction Processing (TPS), Management Information, Decision Support (DSS), Expert Systems, Communication & Collaboration (CCS), Office Automation (OAS).
- **TPS** records daily business transactions and feeds data to other systems.
- **DSS** analyses data to support complex decision-making.
- **Expert System** captures and applies specialised human knowledge to give expert-like advice.
- **CCS** facilitates teamwork and information exchange among stakeholders.
- **OAS** improves office-worker productivity by automating clerical tasks.

2.3 Enterprise Applications

- **ERP** (Enterprise Resource Planning): integrates business processes into one unified system; centralises data and reduces unmanaged risk.
- **SCM** (Supply Chain Management): manages information flow between a business, its suppliers, and partners; optimises logistics and delivery.
- **CRM** (Customer Relationship Management): captures and responds to customer data; directly counters "lack of user involvement."
- **EAI** (Enterprise Application Integration): tools and methods to connect disparate systems (e.g. TPS and DSS) so they work together seamlessly.

2.4 Stakeholders & Development-Process Roles

- A **stakeholder** is anyone with an interest in an existing or proposed system.
- **System owners** provide funding and act as executive advocates.
- **System users** are the "customers" who use the software daily to process and exchange data.
- **Systems analysts** study organisational problems and propose how people, data, processes, and IT can best improve the business.
- **System designers** translate business requirements and constraints into a technical solution (databases, inputs, outputs, screens, networks).
- **System builders** construct the components from the design specifications.
- **Project managers (PM)** plan, monitor, and control schedule, budget, deliverables, and customer satisfaction.
- **External Service Provider (ESP)** sells expertise to other businesses and may act as analyst, designer, or builder.

2.5 System Analyst Skills

- **Problem-solving:** addresses real problems, improves situations, follows new directives.
- **Technical:** working knowledge of IT and programming.
- **Soft skills:** communication, flexibility, ethics.
- **Business knowledge:** to propose effective solutions.

3 The Software Development Process

3.1 Methodology Fundamentals

- A **software development methodology (SDM)** is a formalised set of activities, methods, best practices, deliverables, and automated tools used to build and improve information systems.
- Benefits of a consistent process: lets management shift resources between projects, produces consistent documentation, promotes quality, and reduces lifetime maintenance costs; reduces risk and maintains competitiveness.

3.2 Capability Maturity Model (CMM)

- **Level 1 — Initial:** no prescribed process; unpredictable, poorly controlled.
- **Level 2 — Repeatable:** project-management practices track cost, schedule, functionality.
- **Level 3 — Defined:** a standard methodology is adopted for all projects.
- **Levels 4 & 5 — Managed & Optimizing:** quality goals are set; processes are continuously monitored and improved from data.

3.3 System Life Cycle & High-Level Activities

- Two stages: *systems development* and *operation & maintenance*.
- **Planning & Initiation:** system request, feasibility analysis, work plan — *why*.
- **Analysis:** gather requirements, build system proposal — *what*.
- **Design:** design strategy, architecture, database specs — *how*.
- **Construction & Implementation:** build, test, deliver, train users, document.

3.4 Structured Design Methods

- Traditional, linear/partitioned, phase-by-phase. Best when requirements are well defined.
- **Waterfall:** strictly sequential; poor for unclear requirements or short schedules.

```
Planning --> Analysis --> Design --> Implementation
   (each phase must complete before the next begins)
```

Listing 1: Waterfall — strictly sequential phases

- **Parallel development:** a shared analysis and general design phase, then the project splits into independent subprojects that each run their own design and implementation concurrently; reduces overall delivery time.

```
Planning --> Analysis --> General Design
                                |
              +-----------------+----------------+
              |                 |                |
              v                 v                v
         Sub-Design A     Sub-Design B     Sub-Design C
              |                 |                |
              v                 v                v
           Impl. A           Impl. B          Impl. C
              +-----------------+----------------+
                                |
                                v
                          Integration
```

Listing 2: Parallel Development — split then reunite

- **V-model:** each development phase on the left leg of the V is paired with a corresponding validation/testing phase on the right leg; coding sits at the bottom; testing phases are defined *before* coding begins.

```
Requirements Analysis ---------> Acceptance Testing
         \                                ^
          v                               |
    System Design -----------> System Testing
              \                           ^
               v                          |
         Arch. Design ------> Integration Testing
                   \                     ^
                    v                    |
              Detailed Design --> Unit Testing
                         \           ^
                          v         /
                         Coding / Implementation
      (left leg: definition)   (right leg: verification)
```

Listing 3: V-model — development paired with testing

3.5 Rapid Application Development (RAD)

- Uses prototyping/phased development; runs analysis, design, and implementation concurrently to deliver versions quickly. Excellent for unclear requirements / short schedules.
- **Phased development:** sequential versions; concurrent A/D/I per version; excellent schedule visibility.
- **System prototyping:** a prototype refined by user feedback becomes the final system; poor for complex/highly reliable systems.
- **Throwaway prototyping:** prototype only to clarify requirements, then discard and build properly; best for unfamiliar technology and complex design; excellent reliability.

3.6 Agile Methods

- Short cycles, adaptable to change, simplicity, continuous feedback. Excellent when requirements are very unclear.
- Family includes **Scrum**, **eXtreme Programming (XP)**, and DSDM.

3.7 Methodology Selection (summary table)

See the cheat sheet (Section 27) for the full "Pick Your Poison" comparison.

3.8 Key Deliverables & Cross-Life-Cycle Activities

- Planning ⇒ **Project Plan**; Analysis ⇒ **System Proposal**; Design ⇒ **System Specification**.
- Cross-life-cycle activities (fact-finding, documentation) overlap several phases.
- Fact-finding uses interviews and meetings; ongoing support/maintenance handles errors and new requirements after production.

4 System Analysis

4.1 What System Analysis Does

- A structured problem-solving technique: decompose a system into components to understand how they work together; focus on *what* the system does, independent of technology.
- Studies both weaknesses and potential gains in the current system; output is the blueprint for design and construction.
- Work is captured in a **repository** to keep documentation consistent.
- Key activities: scope definition, problem analysis, requirements analysis, logical design, decision analysis.

4.2 Model-Driven Analysis

- Build visual models of current reality or a proposed vision.
- **Structured analysis:** data flow through processes (Data Flow Diagrams).
- **Information engineering:** data structure (Entity-Relationship Diagrams).
- **Object-oriented analysis:** data + behaviour as objects (UML).

4.3 Requirements Analysis

- **Functional requirements (FRs):** inputs, processes, outputs, stored data, business rules.
- **Non-functional requirements (NFRs):** performance, security, usability, deadlines, constraints.
- **Use cases** model business scenarios for which the system gives a defined response.
- **Timeboxing** prioritises mandatory requirements for early versions, deferring desirable features.

4.4 Requirements Discovery Methods

- Fact-finding: interviews, observation, questionnaires, document sampling, research.
- **Joint Requirements Planning (JRP):** facilitated workshops with owners, users, analysts.
- **Discovery prototyping:** users react to a working-but-incomplete system.
- **Rapid architected analysis:** reverse-engineer existing systems into formal models.

4.5 Problem Analysis

- Understand the problem domain before proposing solutions.
- Use cause-and-effect analysis to trace symptoms to root causes.
- A **context diagram** models how the system interacts with external entities and defines its boundary.
- **Objectives** = measurable success targets; **constraints** = limits on solutions.

4.6 Feasibility Analysis

- **Operational:** does the solution meet business requirements?
- **Technical:** is the technology/expertise available?
- **Economic:** costs vs. tangible and intangible benefits.
- **Schedule:** are deadlines realistic?
- **Legal/Cultural:** regulatory compliance and organisational fit.

4.7 Economic Evaluation Techniques

- **Payback analysis:** time for accumulated benefits to recover costs.
- **Return on Investment (ROI):** lifetime profitability (net benefits / total costs).
- **Net Present Value (NPV):** discounted costs vs. benefits over the system lifetime.
- **Time value of money:** future cash is worth less than present cash; discounting converts future amounts to present value.

$$\text{Present Value: } PV = \frac{FV}{(1+i)^n} \qquad \text{Discount factor} = \frac{1}{(1+i)^n}$$

$$\text{ROI} = \frac{\text{Total benefits} - \text{Total costs}}{\text{Total costs}} \times 100\%$$

$$\text{NPV} = \sum_{t=0}^{n} \frac{B_t - C_t}{(1+i)^t} \qquad \text{(accept the project if NPV} > 0\text{)}$$

4.8 Comparing & Choosing Alternatives

- **Candidate Systems Matrix:** documents similarities/differences between alternative solutions (stakeholder impact, data, processes, communications, technology architecture).
- **Feasibility Analysis Matrix:** ranks alternatives using weighted criteria (operational, technical, economic, schedule, cultural, legal); weighted scores support objective selection.

4.9 System Proposal

- Presents the recommended solution to owners/stakeholders: problem background, requirements summary, feasibility results, alternative comparison, costs, benefits, schedule.
- Delivered as a written report (factual or administrative/results-oriented format), an oral presentation, or both; objective is approval to proceed.

4.10 Creeping Commitment & Risk Reduction

- **Creeping commitment:** feasibility is re-evaluated continuously; decisions are refined as information arrives, preventing premature commitment to weak solutions.
- Analysis reduces technical, operational, and economic uncertainty; modeling turns ambiguity into structured representations; iterative validation supports informed, evidence-based decisions.
- Success factors: clear scope (no creep), root-cause problem analysis, active stakeholder involvement, objective feasibility evaluation, well-structured documentation/repositories.

5 Planning & Project Management

5.1 Why Plan?

- Gives direction, defines goals and scope, organises tasks/timelines/responsibilities, improves use of time/budget/resources, surfaces risks early, and raises the chance of success.

5.2 Project Management Tools

- **Work Breakdown Structure (WBS):** hierarchical decomposition into smaller tasks; tracks duration, progress, and dependencies; built incrementally and iteratively.
- **Task dependencies:** which activities must finish before others begin.
- **Gantt chart:** horizontal bar chart of tasks vs. timeline; monitors schedule.
- **Network diagram / PERT / CPM:** visualise task relationships and find the *critical path*.

5.3 Effort Estimation

- Estimation assigns projected time/effort to tasks; trades off functionality, time, and cost.
- Best estimates come from past, similar projects.
- **Use-case points:** based on technical complexity and environmental factors.
- **Story points:** relative effort, common in agile.

5.4 Work Plan & WBS

- A **work plan** is a dynamic, sequential list of all tasks, updated regularly.
- Derive tasks from a similar past project or directly from the chosen methodology.
- WBS standardises task organisation and helps teams learn from past successes/mistakes.

5.5 Scope Management

- Defines and controls what is in the project; keeps it focused and aligned with goals.
- **Scope creep:** new requirements added after start; disrupts timeline and cost.
- **Techniques:** identify all requirements early, allow only necessary changes, analyse change impact, defer changes as future enhancements, use time boxing.

5.6 Staffing

- Match people's abilities to project needs; determine headcount; motivate the team; distribute tasks; minimise conflict.
- **Deliverable: staffing plan** — number/types of people, reporting structure, roles/responsibilities, plus the **project charter** (objectives and rules).

$$\text{Number of people} = \frac{\text{person-months}}{\text{time to complete (months)}} \qquad \text{Lines of communication} = \frac{n(n-1)}{2}$$

The second formula is why communication overhead grows roughly quadratically as the team grows — hence sub-teams and a reporting structure for large projects.

5.7 Motivation & Conflict

- Motivation has the greatest influence on performance; money alone is a weak motivator.
- Techniques: 20% time for personal projects, peer-to-peer recognition, team ownership ("we" not "I"), interest-driven task choice, equitable compensation, autonomy, trust.
- Reduce conflict via cohesiveness, clear roles, charter-defined work/communication rules, clear plans/procedures, and anticipating competing priorities early.

5.8 Environment, Infrastructure & Documentation

- Choose the right (CASE) tools to boost productivity and centralise information; apply standards consistently; support a centralised repository; enforce workflows.
- Document thoroughly and continuously into an organised project binder; use a standard process (e.g. Unified Process) for templates; never postpone documentation to the end.

5.9 Release Planning

- **Inputs:** product vision, high-level backlog, roadmap, team velocity.
- Review constraints (scope, target date, budget); identify minimum releasable features; groom and map backlog items to sprints; produce a release plan (features, timeline, cost, sprint mapping).
- **Fixed-date planning:** sprint count from the calendar; estimate velocity as a slower–faster *range*; slower $\times$ sprints = "will-have" line; faster $\times$ sprints = "might-have" line.
- **Fixed-date burnup chart:** categorise into will-have / might-have / won't-have; prioritise must-haves before the fixed ship date; track cumulative progress.
- **Fixed-scope burndown chart:** remaining story points vs. schedule, compared with high/low/average velocity, to gauge the chance of completing fixed scope.
- **Fixed-scope burnup chart:** completed points rising toward a fixed target scope.

5.10 Project-Management Software

- Support planning/scheduling/tracking; central platform for tasks, docs, communication; improve visibility into progress/timelines/resources; surface risks/delays/dependencies early.

II Agile & Scrum

6 Agile Fundamentals

6.1 History

- Agile emerged around 1995 as a reaction to rigid processes; early critics saw it as "controlled code-and-fix," which applied to only a small group.
- Builds on design/coding/testing practices that improve flexibility and productivity; keeps explicit but lightweight processes.
- Major methods: DSDM, Scrum, XP, ASD, Crystal family, Feature-Driven Development (FDD), AUP.

6.2 The Agile Manifesto

We value the items on the left more, while still valuing those on the right:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

6.3 The Twelve Principles (condensed)

- Satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late, for competitive advantage.
- Deliver working software frequently (weeks, not months).
- Business people and developers work together daily.
- Build around motivated individuals; give them environment, support, and trust.
- Face-to-face conversation is the most efficient communication.
- Working software is the primary measure of progress.
- Promote sustainable development at a constant pace.
- Continuous attention to technical excellence and good design.
- Simplicity — maximising the work *not* done — is essential.
- Best architectures/requirements/designs emerge from self-organising teams.
- Regularly reflect and tune behaviour to become more effective.

6.4 Benefits, Challenges, and Agile vs. Traditional

- **Benefits:** adaptability via short cycles, faster usable increments, better collaboration, early issue detection, higher customer satisfaction.
- **Challenges:** shifting from traditional management, weak customer engagement, poor estimation, scaling difficulty, superficial "cargo-cult" adoption.
- **Agile vs. traditional:** iterative vs. sequential, evolving vs. fixed requirements, collaboration vs. heavy documentation, continuous adaptation vs. upfront planning, incremental value vs. big-bang delivery.
- *When is Agile wise?* When requirements are unclear/volatile, feedback is available, the team is co-located/empowered, and early partial value is useful. It is a weaker fit for fixed-scope, safety-critical, or heavily regulated big-design-up-front work.

6.5 Agile Metrics

- **Velocity:** work completed per iteration.
- **Burndown chart:** remaining work over time.
- **Cycle time:** start-to-finish duration of a task.
- **Defect rate:** product-quality signal.
- **Customer feedback:** qualitative value signal.

7 Scrum

7.1 Background

- "Scrum" (1986) described fast, flexible product development at Japanese firms (Honda, Canon). Sutherland & Schwaber introduced the software variant in 1995. The name comes from rugby — emphasising teamwork.

7.2 Roles

- **Scrum Master:** ensures the team follows Scrum rules; facilitator and leader, not a project manager; has no authority over the team; explains practices internally and to external stakeholders.
- **Product Owner (PO):** maximises product value; knows the business/customer; sole owner of the product backlog; ensures user stories meet the Definition of Ready; holds the product vision.
- **Developer Team:** cross-functional (define, develop, test, deploy, maintain, communicate); typically 5–9 people; self-organising.

7.3 Artifacts

- **Product Backlog:** continuously evolving, ordered list of requirements; the single source of truth for product modifications.
- **Sprint Backlog:** backlog items selected for the sprint plus the tasks to deliver them (develop, build, review, test); the team's internal prediction for the next increment.
- **Potentially Shippable Product Increment:** delivered each sprint; can ship to production immediately because it has no outstanding tasks.

7.4 States: Definition of Ready vs. Definition of Done

- **Definition of Ready (DoR):** rules a user story must meet before the team accepts it; ensures clarity up front and minimises mid-sprint discussion.
- **Definition of Done (DoD):** criteria for "potentially shippable" (coded, unit + coverage tested, functionally/performance/UAT tested, reviewed, documented). Only work meeting the DoD is delivered.

7.5 Events

- **The Sprint:** fixed timebox (often 1–2 weeks); shorter sprints mean shorter refinement/poker/retrospective sessions.
- **Daily Stand-up (≤15 min):** each member answers (1) what did I accomplish? (2) what will I do next? (3) what is blocking me? It is inspection/synchronisation/adaptive planning — *not* a problem-solving or traditional status meeting.
- **Planning Poker:** quantify effort for new work; the whole team agrees a size; a "common view" forms and sizing reliability (burn rate) improves over time.
- **Sprint Review (product demo):** demo to team, PO, and business/customer; gather and inject feedback for the next sprint.
- **Team Retrospective:** after each sprint, evaluate what went well/badly to improve the way of working.
- **Backlog Refinement:** anticipate/define stories for the next sprint; usually mid-sprint; lets the PO improve stories before the next sprint.

7.6 Social Objects

- **Scrum Board** (e.g. Kanban): activities move left-to-right across columns.
- **Burn-Down Chart:** planned points are "burned" as features complete; target is 0 remaining at sprint end; outlines the burn rate so the team can steer progress.
- **Impediment Board:** external blockers (e.g. "not enough desks," "network down"); the Scrum Master ensures they are handled.

7.7 Scrum Flow

- PO + team define the sprint goal; Planning Poker sizes the work; stories meeting the DoR enter the sprint backlog.
- The sprint runs uninterrupted; the PO may *not* change stories mid-sprint.
- At the end: demo to the customer, run the retrospective, refine the backlog (can also be mid-sprint), and the PO adds updated stories back to the product backlog.

7.8 Benefits

- Delighted customers (value early and often), improved ROI, reduced cost (less waste), fast results (time-boxed sprints), and confidence via transparency/inspection/adaptation.
- **Visibility** reduces misunderstanding and rework; **risk** drops because it becomes clear early whether the team is building the right product; **business value** is generated each sprint by shipping early instead of letting work get "stuck."

8 Requirements in Scrum (User Stories)

8.1 Requirements: Sequential vs. Scrum

- **Sequential:** requirements are fixed, fully detailed up front, standalone, and followed strictly — assuming early stability reduces uncertainty.
- **Scrum:** details emerge through ongoing conversation, expanded just enough to start; stored as flexible **Product Backlog Items (PBIs)** that evolve as the team learns.

8.2 User Stories and the 3 Cs

- User stories express business value clearly for business and technical readers; ideal for features but not for everything (e.g. defects are better as plain descriptions).
- **Card:** the brief written form (index card / sticky note).
- **Conversation:** ongoing discussion that uncovers detail and shared understanding.
- **Confirmation:** acceptance criteria / conditions of satisfaction (often on the card's back); used to build and test the story and to verify correctness.

The standard card template and an example, plus a Gherkin-style way to write the *Confirmation* (acceptance criteria) — this is what "conditions of satisfaction" look like in practice.

```
As a [role], I want [goal], so that [benefit].

Example
  As a registered shopper, I want to save items to a wishlist,
  so that I can purchase them later.

Acceptance criteria (Gherkin: Given / When / Then)
  Given I am logged in and viewing a product
  When I click "Add to wishlist"
  Then the item appears in my wishlist
  And the wishlist count increases by one
```

Listing 4: User-story card + acceptance criteria (added)

8.3 Levels of Detail

- **Epic:** largest story; months long; spans a release or more; never enters a sprint; a placeholder for big work.
- **Theme:** medium; weeks long; a significant functional area; still too large for one sprint.
- **Sprintable (implementable) story:** smallest; days long; fits one sprint; enough detail to build; describes the *what* from the user's view.
- **Task:** below stories; hours long; one or two developers; describes the *how*.

8.4 INVEST — Evaluating Stories

- **Independent:** loosely coupled; minimise (not necessarily eliminate) dependencies.
- **Negotiable:** captures essence and intent; leaves implementation open for discussion.
- **Valuable:** delivers value to a customer and/or user; PO approves purely technical work.
- **Estimatable:** if not, it is too large or ambiguous; may need a spike.
- **Sized appropriately (Small):** fits the sprint it is planned for.
- **Testable:** pass/fail against acceptance criteria.
- *Ready for a sprint?* When it satisfies INVEST *and* the team's DoR: small enough to finish in the sprint, clear and negotiated, estimated, and testable with explicit criteria.

8.5 Non-Functional Requirements & Knowledge-Acquisition Stories

- NFRs are system-wide constraints (e.g. "web browser support") that affect almost all PBIs; fold as many as possible into the DoD so every feature is validated against them.
- **Knowledge-acquisition stories** (prototypes, PoCs, experiments, spikes) gain information; their value must justify their cost — otherwise prefer a fail-fast approach.

8.6 Gathering & Mapping Stories

- **User-story-writing workshops:** PO, Scrum Master, team, stakeholders brainstorm stories top-down (epics → smaller) or bottom-up (sprintable stories for the next release).
- **Story mapping:** decompose a high-level activity into a left-to-right workflow; epics at the top, themes as the task sequence, sprintable stories arranged vertically by priority; gives context across releases.

8.7 PBI Types

- **Feature:** new user-facing capability (user-story format).
- **Change:** adjusts/improves existing functionality.
- **Defect:** a gap between expected and actual behaviour; prioritised by severity/impact.
- **Technical improvement:** refactoring/infrastructure; not user-visible; needs PO approval.
- **Knowledge acquisition:** spikes/PoCs to reduce uncertainty.

8.8 Grooming and the DEEP Model

- Backlog grooming/refinement is continuous: the PO and team clarify, split, estimate, and reorder PBIs so items are ready for upcoming sprints.
- **DEEP:** *Detailed appropriately* (near-term items small/detailed and on top), *Emergent* (never frozen), *Estimated* (effort attached; rough T-shirt sizes for large items), *Prioritized* (ordered by value/urgency).
- **Who/when:** PO leads and decides; the team spends up to ∼10% of sprint time; initial grooming at release planning, ongoing in workshops, after dailies, and during reviews.

8.9 DoR & Release-Flow Management

- The **DoR** is a checklist a PBI must satisfy before sprint selection; a strong DoR raises the chance of meeting the sprint goal.
- Partition the backlog with two lines into **must-have**, **nice-to-have**, and **won't-have** for a release; must-haves are essential and built first.
- **Pipeline management:** keep ∼2–3 sprints of ready stories; too little grooming starves the pipeline, too much creates waste through unnecessary detail and rework.

9 Sprints in Scrum

9.1 Estimation, Units & Velocity

- Estimate total feature size, measure team velocity, then **duration = size / velocity**, which drives cost.
- The development team (who do the work) produce the estimates; the PO clarifies; the Scrum Master facilitates. Keep estimates separate from commitments; favour accuracy over precision; prefer relative sizing.
- **Units:** *story points* (relative magnitude/complexity/effort) or *ideal days* (effort-days; keep distinct from elapsed time).

$$\text{Sprints needed} \approx \frac{\text{total backlog size (points)}}{\text{average velocity (points/sprint)}}$$

9.2 Planning Poker

- Consensus-based discussion that uncovers assumptions and builds shared understanding; groups comparable items into bins on a chosen scale (more granularity at the low end).
- Industry-standard scales: Fibonacci-like `1, 2, 3, 5, 8, 13, 20, 40, 100`, or powers of two.
- Process: PO presents the PBI; team clarifies; each member reveals a private estimate simultaneously (avoiding anchoring); discuss outliers and re-vote until consensus.

9.3 Velocity — Use & Misuse

- Velocity = size of PBIs completed in a sprint (output volume, not value). Used for release forecasting, sprint capacity, and process-health diagnosis.
- Express as a *range*; expect it to rise with focused improvement, then plateau; team changes cause a temporary dip.
- **Don't:** rely on chronic overtime, judge individuals by velocity, or incentivise point inflation. **Do:** prefer sustainable practices and recognise that misuse breeds harmful culture.

9.4 Sprint Planning

- Held at sprint start (∼4–8 h). PO presents goal + prioritised items; the team forecasts what it can deliver; the Scrum Master facilitates and challenges the commitment for realism.
- **Inputs:** groomed product backlog, team velocity, constraints, team capabilities, initial sprint goal.
- **Outputs:** a validated sprint goal and a sprint backlog (the commitment). Break PBIs into tasks of $\leq$8 h.
- **Approach I (1-part):** capacity → refine goal → select PBIs → iterate to capacity.
- **Approach II (2-part):** Part 1 forecast PBIs by points; Part 2 break into tasks + estimate hours, verify hours vs. capacity, adjust, finalise commitment.

9.5 Determining Capacity

- Subtract ceremonies, non-sprint commitments, planned time off, and a risk buffer from total availability; measure in points or effort-hours; forecast by adjusting historical velocity.

9.6 Sprint Execution & Flow

- The team self-organises; the Scrum Master coaches and removes impediments; the PO stays available for feedback and acceptance.
- Minimal up-front task planning (just critical dependencies); avoid heavyweight Gantt charts; plan opportunistically and adapt.
- **Flow:** manage work collectively (not individual utilisation); *swarm* to finish items before starting new ones; adopt a "Musketeer" (all-for-one) attitude; build T-shaped skills; avoid mini-waterfalls that leave partially done work.
- Use information radiators (task board, burndown/burnup); move tasks `to-do` → `in-progress` → `done`.

9.7 Burndown & Burnup Charts

- **Sprint burndown:** plot remaining task-hours daily (hours on y, time on x); slope shows progress; trend-line intersection at sprint end = on schedule; early = can pull in work; late = over-committed/slow ⇒ intervene.
- **Sprint burnup:** plot *completed* work (prefer story points) toward the sprint goal; emphasises delivered value.

9.8 Sprint Review

- Informal, high-value inspect-and-adapt of the increment, at sprint end before the retro; only fully completed work is shown.
- Participants: Scrum team, internal stakeholders, other teams (sales/support), and external customers/users. Roles: Scrum Master facilitates, PO presents, developers demo (rotated).
- Activities: **Demonstrate** (a tool for conversation, use automated test results if non-visual), **Discuss** (defer deep problem-solving), **Adapt** (validate product-market fit; reprioritise/add/remove backlog items; update scope/schedule/budget).
- *Not* a formal sign-off (acceptance happens during execution); turn perceived-incomplete items into new PBIs.

9.9 Sprint Retrospective

- The team inspects its own process/practices/tools/communication. Core questions: what to *continue*, *stop*, and *start*.
- Participants: developers (diverse perspectives), Scrum Master (facilitates), PO (helps optimise flow, attendance depends on team comfort); outsiders by invitation only — protect psychological safety.
- Flow: set the atmosphere (systems not individuals; blame-free), share context (objective data + Event Timeline + Emotions Seismograph), identify insights (silent grouping, root causes), determine actions (clear owners, add improvement tasks to the next sprint backlog), close (recap, appreciate, brief meta-retro).
- **Follow-through:** put improvement tasks in the backlog *before* new features, reduce capacity to account for the effort, and avoid separate improvement plans that get marginalised. *Improvement is not guaranteed right after a retro — it requires the actions to actually be executed and reinforced over subsequent sprints.*

10 eXtreme Programming (XP)

10.1 Overview, Values, Principles

- XP (1996, Kent Beck); first book 1999, revised 2004; framed as an *engineering discipline* but with a clear, structured process.
- **Values:** communication, simplicity, feedback, courage.
- **Principles:** rapid feedback, assume simplicity, incremental change, embrace change, quality work.

10.2 The XP Process (phases)

- **Exploration:** refine stories/priorities; pick next-iteration stories; collaborate with the customer; identify risks; keep planning lightweight.
- **Planning:** split stories into tasks; integrate feedback; write acceptance tests first; create the iteration plan; estimate and commit.
- **Iterations to Release:** continuous review; pair programming + collective code; small frequent releases; testing + analysis; final integrated release.
- **Productionizing:** system-wide verification & validation of the first release; deploy; rollback readiness; smoke testing + monitoring.
- **Maintenance:** implement remaining requirements; bug fixes; iterate phases 2–4 per cycle; integrate production feedback; refactor.
- **Death:** close the project; post-mortem; document lessons; archive; celebrate.
- **Development "engine"** = phases 2–4; each run produces a new release.

10.3 The 12 XP Practices

- Planning Game; Small Releases; Metaphor; Simple Design;
- Testing (tests before code); Refactoring; Pair Programming; Collective Ownership;
- Continuous Integration; 40-Hour Week (sustainable pace); On-Site Customer; Coding Standards.

10.4 System Metaphor & Team

- A **Spike** (small prototype) explores architecture and helps define the **Metaphor** — a simple analogy giving overall architecture without over-constraining.
- **Core team:** Developer/Programmer, Customer, Manager/Tracker, Coach.
- **Optional:** Analyst, Testers, Resource Manager.

III Modeling

11 Requirements & Process Modeling

11.1 Requirements Discovery

- Identifying/extracting system problems and solution requirements from users.
- A **system requirement** describes what the system must do or a property it must have (a.k.a. business requirement).
- **Functional:** specific behaviours — create/update/calculate/validate; inputs, outputs, triggers; business rules; verified by functional test cases.
- **Non-functional:** qualities — performance (response time, throughput, scalability, availability), security (authn/authz, confidentiality, integrity, auditing), operational/business constraints (cost, maintainability, usability, compliance, reliability); verified by metrics, SLAs, security assessments.

11.2 Consequences of Flawed Requirements

- Higher cost, delayed delivery, user dissatisfaction/rejection, escalated maintenance, unreliability/downtime, damaged IT reputation.

11.3 Requirements Documents & Management

- **Requirements Definition Document:** formal communication of requirements; acts as a contract; a.k.a. Requirements Specification / Functional Specifications.
- **Requirements management:** ongoing handling of changes; up to 50%+ of requirements may change before deployment.

11.4 Fact-Finding Methods

- Document/form/database sampling; research and site visits; observation of the work environment; questionnaires; interviews; prototyping; Joint Requirements Planning (JRP).

11.5 Use-Case Modeling

- User-centred; captures requirements from the perspective of user interactions; specified in stakeholder-understandable form; supports verification/validation.
- **Use case:** a behaviourally related sequence of steps for one business task (automated + manual).
- **Use-case diagram:** actors, use cases, and relationships; shows system boundary and who interacts how.
- **Use-case narrative:** detailed textual description — inputs, outputs, step-by-step activities.
- *Benefits:* elicits correct requirements, shared vocabulary with users, scope clarity, basis for estimation/testing.

11.6 Use-Case Symbols, Actors & Relationships

- **Use case:** horizontal ellipse. **Actor:** human, organisation, system, device, or *time* (temporal events).
- **Four actor types:** primary business actor (chief beneficiary), primary system actor (initiates the event), external server actor (responds to a request), external receiver actor (receives value but isn't primary).
- **Association:** solid line; arrowhead → use case means the actor initiated it; no arrowhead = receiver; may be uni- or bidirectional.
- **Extends (⟨⟨extends⟩⟩):** optional steps extracted to simplify a base use case; arrow from extension → base.
- **Uses/Includes (⟨⟨uses⟩⟩):** common steps factored into an abstract use case invoked by others; arrow from base → abstract.
- **Depends On (⟨⟨depends on⟩⟩):** which use cases must occur first; sets development sequence.
- **Inheritance:** shared actor behaviour generalised to an abstract actor that others inherit from.

11.7 Process of Requirements Use-Case Modeling

- Step 1: identify business actors. Step 2: identify business use cases. Step 3: construct the use-case model diagram (associations, uses, extends, depends-on). Step 4: document use-case narratives (triggers, steps, outcomes; normal + alternative flows).
- Goal: a user-perspective model of *what* is required (no implementation detail) that supports estimation/scheduling.

A use-case diagram and a narrative can be written textually (PlantUML), which is handy for versioning models in git alongside code.

```
@startuml
left to right direction
actor Customer
actor "Kitchen Staff" as Kitchen

rectangle "Los Pollos Hermanos Ordering System" {
  Customer --> (Place Order)
  (Place Order) ..> (Apply Promo) : <<extends>>
  (Place Order) ..> (Validate Payment) : <<uses>>
  (Place Order) --> Kitchen
}
@enduml
```

Listing 5: Use-case model as PlantUML (added)

```
Use Case:         Place Order
Primary Actor:    Customer
Trigger:          Customer selects items and confirms
Preconditions:    Menu is available; customer is identified
Main Flow:
  1. Customer selects menu items
  2. System calculates total (and applies promo if eligible)
  3. Customer submits payment
  4. System validates payment and creates the order
  5. System notifies kitchen staff
Postconditions:   Order is queued; receipt is issued
Alternate Flow:   3a. Payment fails -> system prompts retry / cancel
```

Listing 6: Use-case narrative skeleton (added)

11.8 Modeling Categories

- **Process model:** logic of business processes/operations; tools include Data Flow Diagrams (DFDs) and Business Process Models (BPMs).
- **Data model:** logical data structure — entities, attributes, relationships; Entity-Relationship Diagram (ERD); supports normalisation/integrity and DB design.
- **Object model:** objects encapsulating data + behaviour; class/sequence/collaboration diagrams; bridges OO analysis to implementation.

12 UML Behavioral Diagrams

12.1 What UML Is

- Unified Modeling Language — a standardised visual language for system structure and behaviour, used in analysis, design, and documentation.
- Open standard from the Object Management Group (OMG), also known for CORBA; introduced 1997; unified competing OO methods. UML standardises *modeling*, not a development process.
- In Agile it is used selectively (often whiteboard sketches) when it adds value; with waterfall, models become formal documents passed between phases.

12.2 UML in Analysis, Design & Documentation

- **Analysis:** use case, conceptual class, activity, state diagrams.
- **Design:** class, sequence, package, state, deployment diagrams.
- **Documentation:** simpler diagrams that explain structure/behaviour of a finished system to aid maintenance.
- **Categories:** *structural* (class, package, deployment) vs. *behavioral* (use case, sequence, activity, state).

12.3 Use Case Diagram

Models interactions between external actors and the system; defines the system boundary; relationships include include, extend, and generalization.

12.4 State Machine Diagram

- Shows an object's states over its life cycle; **transitions** (triggered by events/conditions) move between states; a **guard** must hold for a transition; an **action** executes during a transition.
- **Special states:** initial, final, composite (with substates), entry/exit actions.
- **Superstates** group related substates (transitions can target the superstate; substates inherit common transitions) — reduces repetition.
- **Concurrent states:** parallel regions operate simultaneously; events may trigger transitions in each region independently.
- Best for event-driven/reactive, state-dependent behaviour.

12.5 Activity Diagram

- Models workflow: activities (tasks), control flows (order), decision nodes (branching), initial/final nodes, objects & object flows (data).
- **Action decomposition** breaks a complex activity into smaller actions; **signals** are asynchronous messages that can start/influence activities.
- **Partitions (swimlanes)** assign activities to actors/roles/components.
- Use for use-case behaviour, business processes (human + system), decisions, parallelism.

12.6 Sequence Diagram

- Shows object interactions over time: lifelines (participants), messages (arrows), activation bars (operation duration); vertical axis = time.
- Models object creation (message begins a new lifeline) and deletion (termination marker).
- **Loops** repeat message sets; **alt** fragments express conditional paths.
- Derived from important use-case scenarios; useful in analysis and design.

A sequence diagram is also concise as PlantUML, with a loop and an alt fragment shown.

```
@startuml
actor User
participant UI
participant API
participant DB

User -> UI : submit login
UI -> API : POST /login
loop until valid or 3 tries
  API -> DB : find user
  alt credentials valid
    DB --> API : user record
    API --> UI : 200 + token
  else invalid
    DB --> API : not found
    API --> UI : 401
  end
end
@enduml
```

Listing 7: Sequence diagram as PlantUML (added)

12.7 Other Behavioral Diagrams & BPMN

- **Communication** (interactions emphasising relationships), **interaction overview** (activity + interaction), and **timing** diagrams (state/condition over time).
- **BPMN** (Business Process Model and Notation): graphical standard for business processes — events, activities, gateways (decisions), sequence flows, pools & lanes (participants/responsibilities). Complements UML by focusing on business rather than software structure; bridges technical and business stakeholders.

13 UML Structural Diagrams

13.1 Class Diagrams

- The most common UML structural diagram; models the static structure of a system by showing classes, attributes, and methods.
- Depicts relationships: inheritance, association, dependency.
- Serves as a blueprint for OO design and code generation; used from analysis through implementation.
- **Elements:** classes (rectangles divided into name/attributes/operations); attributes have visibility (`+` public, `-` private, `#` protected); multiplicity labels (1, 0..*, 1..*); abstract classes and interfaces shown in italics or with `<<interface>>`.
- **When to use:** requirements analysis (domain concepts), detailed design (before coding), documentation, code generation, communication of hierarchies and patterns.

13.2 Class Diagram Relationships

- **Association:** structural relationship (solid line), with multiplicity and optional role names (e.g. "employer" / "employee") and navigation arrows (uni- or bidirectional).
- **Operations:** listed in the third compartment; format: `visibility name(params): returnType`; abstract operations in italics.
- **Generalization (inheritance):** hollow triangle arrow pointing from child to parent; child inherits all members and can add/override; represents an "is-a" relationship.
- **Dependency:** dashed arrow from dependent to provider; local variable use, method parameter, static call; weaker than association; stereotypes: `<<use>>`, `<<create>>`, `<<call>>`.
- **Abstract class vs. interface:** abstract classes can have concrete methods and cannot be instantiated; interfaces define a pure contract (all methods abstract); a class can implement multiple interfaces but inherits from only one abstract class.

```
@startuml
abstract class Vehicle {
  +speed: int
  +{abstract} move(): void
}
class Car {
  +fuelType: string
  +move(): void
}
Vehicle <|-- Car          ' generalization (hollow triangle)
Car "1" -- "0..*" Wheel   ' association with multiplicity
@enduml
```

Listing 8: Class diagram fragment as PlantUML

13.3 Component Diagrams

- Models the high-level organisation of software components: modular, replaceable parts that encapsulate behaviour and implement interfaces.
- Shows how components wire together through provided and required interfaces; focuses on dependencies and interactions, not internal details.
- **Elements:** components (`<<component>>` stereotype); provided interfaces (lollipop: circle on a line); required interfaces (socket: half-circle); assembly connector (ball-and-socket); dependency (dashed arrow).
- **When to use:** high-level architecture partitioning, deployment/configuration of third-party or legacy components, communicating structure to multiple teams, planning integration testing.

13.4 Composite Structure Diagrams

- Shows the *internal* structure of a classifier (class or component): how parts collaborate and connect internally.
- Unlike class diagrams, focuses on *runtime* configuration of interconnected instances (the "white-box" view).
- **Elements:** parts (rectangles within the main box), ports (small squares on boundary managing incoming/outgoing requests), connectors (communication paths), interfaces (lollipop/ socket on ports), collaborations (dotted ellipses showing interaction patterns).
- **When to use:** modelling internal wiring of complex components, specifying design patterns (MVC, Observer), documenting product-line assemblies with predefined connections.

13.5 Deployment Diagrams

- Models the physical distribution of software artifacts across hardware nodes; shows what runs where in a deployed system.
- Essential for understanding system topology, redundancy, and network communication.
- **Elements:** nodes (3D boxes — `<<device>>` for physical hardware, `<<execution environment>>` for software platform like OS or web server); artifacts (actual files: .jar, .dll, .exe) shown inside nodes; communication paths (solid lines between nodes indicating network connections).
- **When to use:** planning hardware allocation, documenting multi-tier architectures, identifying single points of failure, planning redundancy, deploying to cloud/Kubernetes, performance analysis.

IV Software Design

14 Introduction to System Design

14.1 What System Design Is

- System design focuses on the implementation of requirements identified during analysis.
- The primary goal is to create a technical blueprint that guides the actual development process.
- Design activities define data storage strategies and physical system architecture.
- User interface components and input/output mechanisms are established at this stage.
- Analysis and design are iterative processes that require frequent refinement and cross-functional feedback.

14.2 From Analysis Models to Design Models

- **Verify & validate:** cross-reference activity diagrams with use-case descriptions; apply use-case realisation to trace functional requirements through the design; balance functional, structural, and behavioural model dimensions.
- **Evolve:** expand on analysis models to incorporate non-functional requirements (performance, processing distribution, environmental constraints, UI, database configurations).
- Primary design objectives: maintainability, cost-effectiveness, efficiency, and reliability.
- Implementation techniques: factoring, partitioning, collaborations, and architectural layering.

14.3 Design Strategy Decision

- Options: custom in-house development, purchasing packaged software, or outsourcing.
- Evaluation: identify tools/skills needed internally, or locate suitable packages; build alternative matrices of pros/cons.
- Feasibility must incorporate technical, economic, and organisational dimensions.
- Requests for Proposals (RFP) or Information (RFI) secure vendor cost and time estimates.

14.4 Software Architecture

- The high-level structure of a system and the discipline of creating such structures.
- Defines fundamental components, their responsibilities, and the relationships and interactions between them.
- Addresses non-functional requirements: scalability, security, performance, reliability.
- Acts as a bridge between business requirements and technical implementation.
- Decisions made at this level are *difficult and costly to change later* — the most critical phase of the design lifecycle.
- Common architectural patterns: layered, client-server, microservices, event-driven.

14.5 Architecture Characteristics

High-level qualities that shape system behaviour and trade-offs.

- **Availability & Reliability:** uptime percentage; correct outputs under stated conditions; achieved via redundancy, monitoring, failure mitigation (health checks, retries, circuit breakers).
- **Testability & Scalability:** testability uses dependency injection, mocks, contract testing, observability hooks; scalability handles increased load by adding resources (horizontal vs. vertical; stateless designs aid scaling).
- **Security & Agility:** CIA triad (confidentiality, integrity, availability); threat modelling, authn/authz, least privilege, encryption (at rest and in transit); agility enables quick pivots via Agile practices, CI/CD, and modular architecture — DevSecOps makes security *enable* agility, not block it.
- **Fault Tolerance & Elasticity:** fault tolerance keeps the system operating despite failures (redundancy, replication, graceful degradation, failover); elasticity auto-scales resources based on real-time demand — essential in cloud environments.
- **Recoverability & Performance:** recoverability restores service after failure (RTO/RPO, backups, disaster recovery, point-in-time recovery); performance covers responsiveness, throughput, latency, resource usage (caching, indexing, async processing, CDNs); fast recovery (low RTO) often requires performance investments such as hot standbys.
- **Deployability & Learnability:** deployability covers ease and safety of releasing changes (CI/CD, blue/green and canary deployments, feature flags); learnability measures how quickly new developers/operators can understand and use the system (clear docs, consistent APIs, good logging/monitoring); high deployability supports rapid learning via fast feedback cycles.

14.6 Trade-offs, Technical Debt, and Continuous Improvement

- Every architectural decision creates trade-offs between competing quality attributes (e.g. more security $\Rightarrow$ more latency).
- Architects prioritise requirements based on stakeholder needs and business objectives; document the rationale behind structural choices.
- **Architectural drift** occurs when the actual implementation deviates from the planned design; manage via technical debt tracking and periodic architecture reviews.
- Continuous monitoring of system metrics informs future architectural updates.

14.7 Architecture vs. Design

- **Architecture** defines the system's high-level structure: major components/subsystems, their responsibilities, interfaces, and interactions; driven mainly by *non-functional* requirements and constraints; system-wide.
- **Design** defines the detailed solution inside those components: classes, methods, algorithms, data structures, UI details, and database schema; driven mainly by *functional* requirements; component/module-level.
- Architecture sets constraints and guidelines; design works within those boundaries.
- **Architecture outputs:** architectural style/pattern, component diagram, deployment diagram, technology stack, integration approach, data ownership boundaries.
- **Design outputs:** class diagrams, sequence diagrams, DB tables, UI wireframes, validation rules, error handling logic.
- Example: Architecture chooses a web app with a REST API & MySQL; design specifies the classes, logic, queries, and UI flows.

14.8 Subsystems, Interfaces, and Packages

- **Subsystems** are the primary decomposition mechanism; the goal is to maximise independence between them, reducing coupling and enabling parallel and independent deployment. Partitions represent closely collaborating class groups; stronger coupling indicates they belong in the same partition.
- **Interfaces** hide subsystem implementation details, exposing only what is needed; the *Facade* pattern provides a simple interface to a complex subsystem; layering organises subsystems into cohesive layers (presentation, business logic, utility) with dependencies flowing only downward.
- **Package diagrams** group similar/related components (use cases, class diagrams); show packages and their relationships (aggregation, association, dependency); changes to packages with dependencies must be handled carefully.
- **Deployment diagrams** model the physical deployment of artifacts onto nodes: artifacts = real-world things (source files, executables); nodes = hardware or execution environments. Two node types: `<<device>>` (physical hardware, e.g. a PC or server) and `<<execution environment>>` (software platform, e.g. Apache).

14.9 The Design: Classes, Units, Modules, UI

- **Design classes** refine analysis classes with specific attributes, methods, and implementation details.
- **Units** are the smallest testable pieces of code (individual methods or functions).
- **Modules** group related design classes and units into cohesive, loosely coupled blocks.
- The **user interface** sits as a separate layer, mediating between users and the underlying domain logic; well-structured design keeps UI concerns separate from business logic.

15 Architectural Patterns

15.1 MVC (Model-View-Controller)

- **Model:** data, business logic, rules (no UI logic). **View:** presentation. **Controller:** turns input into commands for Model/View.
- *Passive model* knows nothing of Views/Controllers; *active model* notifies Views via the Observer pattern.
- **Use when** UI and business logic must be separated, multiple views show the same data, or logic must be testable apart from the UI (Rails, Django, Spring MVC, ASP.NET MVC, UIKit, Android).
- **Watch for** the "fat controller," tight View–Controller coupling; variants MVP and MVVM. Not ideal for real-time tight View/Model sync.

```python
class Model:                        # data + logic
    def __init__(self): self.count = 0
    def increment(self): self.count += 1

class View:                         # presentation only
    def render(self, count): print(f"Count = {count}")

class Controller:                   # input -> model -> view
    def __init__(self, m, v): self.m, self.v = m, v
    def on_click(self):
        self.m.increment()
        self.v.render(self.m.count)
```

Listing 9: Minimal MVC (added)

15.2 MVVM (Model-View-ViewModel)

- **ViewModel** sits between View and Model, exposing observable properties and commands the View *binds* to; the View has no logic.
- Relies on **two-way data binding** and the command pattern; popularised by WPF, now in Angular, Vue, SwiftUI, Jetpack Compose.
- **Use when** the framework supports declarative UI/binding and you want minimal code-behind and testable UI logic.
- **Watch for** silent binding failures, performance with large datasets, bloated ViewModels, and a steeper learning curve.

15.3 Layered (n-tier) Architecture

- Horizontal layers, each a single responsibility; dependencies flow *downward*.
- Typical four layers: **Presentation** (UI/APIs), **Business** (logic/validation/ workflows), **Persistence** (data access/ORM), **Database** (SQL/NoSQL/files).
- **Use when** you need clear separation, parallel team work, or swappable layers; good starting point before microservices (Spring Boot, ASP.NET Core, FastAPI).
- **Watch for** the "sinkhole" anti-pattern (pass-through layers with no logic), added complexity/latency, layer-skipping, and changes rippling across layers.

15.4 Clean Architecture

- Concentric circles; the **dependency rule** points inward toward business logic; outer circles depend on inner, never the reverse (Robert C. Martin; evolves Hexagonal/Onion).
- Layers inside-out: **Entities** (enterprise rules) → **Use Cases** (application rules) → **Interface Adapters** (controllers/presenters/gateways) → **Frameworks & Drivers**. Inner layers define interfaces, outer layers implement them (*dependency inversion*).
- **Use when** business logic must outlive frameworks, decisions on DB/UI must be deferred, and testability/-maintainability are paramount.
- **Watch for** boilerplate/indirection, overkill on small projects, and dogmatic over-abstraction.

15.5 Microkernel (Plugin) Architecture

- A minimal stable **core** plus independent, hot-swappable **plugins**; a **plugin registry** tracks capabilities; a **contract/API** defines core–plugin communication.
- **Use when** features/third-party extensions are added often, experimental features must be isolated, or different customers need different feature sets (Eclipse, Chrome, payment gateways, antivirus).
- **Watch for** getting the core size right, plugin versioning/conflicts, communication overhead, sandboxing/security, and combinatorial testing.

15.6 Microservices

- Decompose into small independent services around business capabilities; each runs in its own process and communicates over the network (REST/gRPC/messaging) and owns its database.
- Supporting pieces: API gateway, service discovery, infrastructure automation (containers, orchestration, CI/CD).
- **Use when** large teams need independent deploys, services scale differently, or a monolith has outgrown itself.
- **Watch for** distributed-system complexity (latency, partial failure, eventual consistency), distributed data/-transactions, and operational overhead. *Start with a monolith; split when necessary.*

16 Design Patterns (Gang of Four)

- Recurring problems get recurring, reusable solutions; documented patterns cut design time and ease understanding/maintenance/extension.
- Three categories: **Creational** (object creation), **Structural** (composing classes/objects), **Behavioral** (communication & responsibility).

16.1 Creational Patterns

Abstract Factory

Creates *families* of related objects through abstract products; clients stay independent of concrete classes. Use for themes/platforms/variants; downside: adding a new product *type* is hard.

```
from abc import ABC, abstractmethod
class Button(ABC):
    @abstractmethod
    def paint(self): ...
class WinButton(Button):
    def paint(self): return "Windows button"
class MacButton(Button):
    def paint(self): return "macOS button"

class GUIFactory(ABC):
    @abstractmethod
    def create_button(self) -> Button: ...
class WinFactory(GUIFactory):
    def create_button(self): return WinButton()
class MacFactory(GUIFactory):
    def create_button(self): return MacButton()

def build_ui(factory: GUIFactory):      # client uses abstractions only
    return factory.create_button().paint()
```

Listing 10: Abstract Factory (added)

Builder

Constructs a complex object step by step, separating construction from representation; the same process can yield different results. Use for many optional parts; avoids "telescoping" constructors.

```
class Pizza:
    def __init__(self): self.toppings = []
class PizzaBuilder:
    def __init__(self): self.pizza = Pizza()
    def add(self, t): self.pizza.toppings.append(t); return self   # fluent
    def build(self): return self.pizza

pizza = PizzaBuilder().add("cheese").add("mushroom").build()
```

Listing 11: Builder (added)

Factory Method

Defines an interface for creating an object but lets subclasses decide the class to instantiate. Use when a class can't anticipate the type it must create; supports the Open–Closed Principle.

```
from abc import ABC, abstractmethod
class Logistics(ABC):
    @abstractmethod
    def create_transport(self): ...      # the factory method
    def plan(self): return self.create_transport().deliver()
class RoadLogistics(Logistics):
    def create_transport(self): return Truck()
```

```
class Truck:
    def deliver(self): return "deliver by truck"
```

Listing 12: Factory Method (added)

Prototype

Creates new objects by *cloning* an existing one, preserving complex state, without depending on concrete classes. Use when creation is expensive; mind deep vs. shallow copy and cycles.

```
import copy
class Document:
    def __init__(self, content): self.content = content
    def clone(self): return copy.deepcopy(self)   # deep copy avoids shared state
template = Document(["header", "body"])
report = template.clone(); report.content.append("footer")
```

Listing 13: Prototype (added)

Singleton

Ensures one instance with a global access point. Use for shared config/logging. Downsides: global state, harder testing, possible SRP violation, concurrency hazards.

```
import threading
class Config:
    _instance, _lock = None, threading.Lock()
    def __new__(cls):
        with cls._lock:
            if cls._instance is None:
                cls._instance = super().__new__(cls)
        return cls._instance
```

Listing 14: Singleton, thread-safe (added)

16.2 Structural Patterns

Adapter

Wraps an incompatible interface to match what the client expects, reusing existing code unchanged. Use for legacy/third-party integration.

```
class OldPrinter:                       # adaptee (incompatible)
    def print_text(self, s): return f"OLD:{s}"
class Printer:                          # target interface
    def output(self, s): ...
class PrinterAdapter(Printer):
    def __init__(self, old): self.old = old
    def output(self, s): return self.old.print_text(s)  # translate
```

Listing 15: Adapter (added)

Bridge

Separates an abstraction from its implementation so both vary independently (composition over inheritance); avoids class explosion across two dimensions of variation.

Composite

Treats individual objects and groups uniformly via a common interface; models part–whole trees (file systems, menus). Mind interfaces that don't fit all node types.

```
class Node:
    def size(self): ...
class File(Node):
    def __init__(self, n): self.n = n
    def size(self): return self.n
class Folder(Node):
    def __init__(self): self.children = []
    def size(self): return sum(c.size() for c in self.children)  # recursive
```

Listing 16: Composite (added)

Decorator

Adds responsibilities dynamically by wrapping an object that shares its interface; a flexible alternative to subclassing. Order of decorators matters.

```
class Coffee:
    def cost(self): return 2.0
class MilkDecorator:
    def __init__(self, c): self.c = c
    def cost(self): return self.c.cost() + 0.5
drink = MilkDecorator(MilkDecorator(Coffee()))   # stackable -> 3.0
```

Listing 17: Decorator (added)

Facade

Provides a simple interface over a complex subsystem; reduces coupling and exposes only what clients need. The subsystem stays accessible for advanced use.

```
class Facade:
    def __init__(self, a, b, c): self.a, self.b, self.c = a, b, c
    def start(self):                # one call hides subsystem orchestration
        self.a.init(); self.b.load(); return self.c.run()
```

Listing 18: Facade (added)

Flyweight

Minimises memory by sharing *intrinsic* state and passing *extrinsic* state externally; a factory reuses instances. Use for huge numbers of similar objects.

Proxy

A placeholder controlling access to a real object with the same interface; enables lazy creation, access control, caching, logging, or remoting.

```
class Image:
    def __init__(self, path): self.path = path; self._load()
    def _load(self): print("loading", self.path)
    def draw(self): return self.path
class ImageProxy:
    def __init__(self, path): self.path, self._img = path, None
    def draw(self):
        if self._img is None: self._img = Image(self.path)  # on demand
        return self._img.draw()
```

Listing 19: Proxy, lazy loading (added)

16.3 Behavioral Patterns

Chain of Responsibility

Passes a request along a chain of handlers until one handles it; sender stays decoupled from receivers. Use for approval flows, event handling, middleware. Risk: a request may go unhandled.

```
class Handler:
    def __init__(self, nxt=None): self.nxt = nxt
    def handle(self, lvl):
        if self.nxt: return self.nxt.handle(lvl)
class Manager(Handler):
    def handle(self, lvl):
        return "approved" if lvl <= 1 else super().handle(lvl)
class Director(Handler):
    def handle(self, lvl):
        return "approved" if lvl <= 3 else "rejected"
chain = Manager(Director())
```

Listing 20: Chain of Responsibility (added)

Command

Encapsulates a request as an object so it can be stored, queued, logged, or undone; decouples invoker from receiver. Use for menus/buttons, queues, undo/redo.

```
class AddText:
    def __init__(self, doc, text): self.doc, self.text = doc, text
    def execute(self): self.doc.append(self.text)
    def undo(self): self.doc.pop()
history = []
def run(cmd): cmd.execute(); history.append(cmd)
def undo(): history.pop().undo()
```

Listing 21: Command with undo (added)

Interpreter

Represents a small language's grammar as classes that interpret expressions (terminal + non-terminal) with a shared context. Good for simple DSLs/rule engines; class count grows fast.

Iterator

Traverses a collection sequentially without exposing its internals; supports multiple/independent traversals. Use to hide structure and standardise iteration.

```
class Tree:
    def __init__(self, items): self.items = items
    def __iter__(self):            # Python's iterator protocol
        for x in self.items: yield x
for node in Tree([1, 2, 3]): print(node)
```

Listing 22: Iterator (added)

Mediator

Centralises communication between objects so they don't reference each other directly; reduces coupling. Use for complex UIs. Risk: the mediator becoming a "god object."

Memento

Captures and restores an object's internal state without breaking encapsulation; Originator creates mementos, a Caretaker stores them. Use for undo/checkpoints; mind memory cost.

```
class Editor:
    def __init__(self): self.text = ""
    def save(self): return self.text          # memento (snapshot)
    def restore(self, m): self.text = m
caretaker = []
e = Editor(); e.text = "v1"; caretaker.append(e.save())
e.text = "v2"; e.restore(caretaker.pop())      # back to "v1"
```

Listing 23: Memento (added)

Observer

Defines a one-to-many dependency: when the subject changes, all observers are notified automatically. Use for events/publish–subscribe and reactive UIs; watch for update storms and leaks from unmanaged subscriptions.

```
class Subject:
    def __init__(self): self._obs = []
    def subscribe(self, o): self._obs.append(o)
    def notify(self, data):
        for o in self._obs: o.update(data)
class Display:
    def update(self, data): print("updated:", data)
```

Listing 24: Observer (added)

State

Lets an object change behaviour when its state changes by delegating to state objects; replaces large conditionals. Use for finite state machines/lifecycles.

```
class Draft:
    def publish(self, doc): doc.state = Published()
class Published:
    def publish(self, doc): print("already published")
class Document:
    def __init__(self): self.state = Draft()
    def publish(self): self.state.publish(self)
```

Listing 25: State (added)

Strategy

Encapsulates interchangeable algorithms behind a common interface, selectable at runtime; composition over inheritance; supports Open–Closed.

```
class Context:
    def __init__(self, strategy): self.strategy = strategy
    def run(self, data): return self.strategy(data)   # plug in any algorithm
quick = Context(sorted)
reverse = Context(lambda d: sorted(d, reverse=True))
```

Listing 26: Strategy (added)

Template Method

Fixes an algorithm skeleton in a base class while subclasses customise specific steps (and optional hooks). Reuses common workflow; relies on inheritance.

```
from abc import ABC, abstractmethod
class Report(ABC):
    def generate(self):            # the template method (fixed sequence)
        self.header(); self.body(); self.footer()
    def header(self): print("=== Report ===")
    @abstractmethod
```

```
    def body(self): ...             # subclasses fill the variable step
    def footer(self): print("--- end ---")
```

Listing 27: Template Method (added)

Visitor

Separates operations from the objects they act on; add new operations without changing element classes (double dispatch). Use when the object structure is stable but operations change often; adding a new element type forces changes to every visitor.

17 System Design: An Incremental Approach

17.1 Client, Server, and DNS

- **Client** requests services/resources; **server** listens and provides them. Separation of concerns: clients handle presentation and user interaction; servers manage data, logic, and persistence. Each tier can scale, evolve, and be secured independently.
- **DNS** maps domain names to IP addresses; decouples human-readable names from physical server locations; enables load distribution (returning different IPs per region), redundancy (one name → multiple IPs), and service mobility (change IP without changing how users reach you).

17.2 Databases

- Stores, retrieves, and manages data persistently; typically a separate node for independent scaling.
- **SQL (relational):** table-based, strict schema, ACID transactions, complex joins (PostgreSQL, MySQL). Choose when you need predictable, structured data.
- **NoSQL (non-relational):** document/graph/key-value/column, schema-flexible, horizontal scaling, high write throughput, unstructured data (MongoDB, Cassandra). Choose for logs, sessions, catalogs, or write-heavy workloads.
- Many system designs use both: SQL for transactions/user accounts, NoSQL for logs/sessions.

17.3 Scaling

- **Vertical scaling (scale up):** add more CPU/RAM/storage to one server. Simpler but has hardware limits and high unit cost.
- **Horizontal scaling (scale out):** add more servers. Near-infinite scaling and better fault tolerance, but adds complexity. SQL typically scales vertically; NoSQL is designed for horizontal scaling.
- **Load balancers** distribute incoming traffic across multiple servers, preventing any single server from being overwhelmed, hiding failures, and routing around unhealthy instances. Common algorithms: round robin, least connections, IP hash, weighted. Sit between clients and servers.
- **Database replication** copies data from a master to one or more slaves; master handles writes; slaves handle reads; a slave can be promoted to master on failure (high availability).

17.4 Caching and CDN

- **Cache:** temporarily stores frequently accessed data for faster retrieval (memory instead of disk/DB); reduces latency and load on downstream services. Strategies: read-through, write-through, write-around, write-back. Examples: Redis, Memcached, browser cache, CDN cache.
- **Cache considerations:** cache what is frequently read and rarely changed (user sessions, product catalogs); set a TTL; handle invalidation carefully (stale data = consistency issues); watch for cache stampede (many simultaneous misses) and plan for warm-up.
- **CDN:** distributed network of servers delivering content based on geographic location; caches static assets (images, CSS, JS, video) at edge nodes worldwide; reduces latency by serving from the nearest server; offloads traffic and bandwidth from the origin; advanced CDNs also accelerate dynamic content and API caching.

17.5 Stateful vs. Stateless Architecture

- **Stateful:** session data on the server that handled the request; requires sticky sessions (same server per user); complicates load balancing, failover, and scaling.
- **Stateless:** no session data on the server; each request carries all needed information; any server can handle any request; load balancing is trivial; scaling is easy (just add/remove servers); session data is stored externally (DB, Redis, or client-side token like JWT).

17.6 GeoDNS and Message Queues

- **GeoDNS:** returns different IP addresses based on the requester's geographic location; directs users to the nearest/most appropriate region; enables content localisation and routing around regional outages. Works alongside CDNs and load balancers.
- **Message queues:** temporary storage buffers for asynchronous communication between services; producers send without waiting; consumers pull at their own pace; decouples services, improves fault tolerance, handles traffic spikes. Examples: RabbitMQ, Apache Kafka, Amazon SQS, Redis Pub/Sub.
- **Use cases:** async processing (sending emails), traffic spike absorption, parallel distribution of work across workers, reliable delivery with retries and dead-letter queues.

17.7 Observability and Supporting Infrastructure

- **Logging:** records discrete events (errors, requests, state) for debugging and audit trails.
- **Metrics:** numerical measurements (latency, throughput, error rate) for monitoring and alerting.
- **Automations:** trigger actions based on logs/metrics (auto-scaling, restart failed services); reduce manual intervention and enable self-healing.
- Common stack: ELK (Elasticsearch, Logstash, Kibana) for logs; Prometheus + Grafana for metrics.
- **API gateway:** single entry point for all client requests; handles routing, authentication, rate limiting, logging, and response aggregation; can aggregate responses from multiple services. NGINX is a widely used high-performance API gateway and reverse proxy.
- **Object storage:** stores data as objects (data + metadata + unique ID) rather than files or blocks; ideal for unstructured data (images, videos, backups, logs); scales horizontally to petabytes with built-in redundancy. MinIO is a popular S3-compatible implementation.

18 UI/UX Design

18.1 UI vs. UX

- **UI (User Interface):** the visual elements users interact with — buttons, icons, colours, typography, layouts. About how something *looks*.
- **UX (User Experience):** the overall feeling and ease of use when interacting with a product. About how something *works and feels*.
- Good UI makes the product attractive; good UX makes it usable and efficient.
- UX includes research, user flows, wireframing, prototyping, and usability testing *before* any visual design begins.

18.2 What Makes Good UX?

- Good UX is *invisible* — users accomplish goals without noticing the interface.
- **Clarity:** users immediately understand what to do and where to go.
- **Efficiency:** common tasks take minimal time and effort.
- **Consistency:** similar actions work similarly across the product.
- **Feedback:** every user action receives a clear response (visual, audible, or haptic).

18.3 The UX Design Process

- UX Design creates products delivering meaningful and relevant experiences by understanding user needs, behaviours, and pain points through research; covering the full journey from first discovery to long-term use (information architecture, interaction design, usability testing, iteration).
- Process phases: **Requirements** (user needs, business goals, constraints) → **Design** (wireframes, prototypes, user flows) → **Verification** (testing with real users) → **Implementation** (developer handoff and collaboration) → **Maintenance** (feedback, usage data, iteration).
- **Sketch** (quick low-fi, ideas) → **Design** (high-fi, colours/typography/ spacing) → **Prototype** (interactive, clickable, simulates real flows). Each stage increases fidelity and catches issues earlier.
- **Wireframes vs. sketches:** sketches are hand-drawn, divergent (many ideas); wireframes are digital, structured, convergent (refine one idea); wireframes use grayscale and placeholders to avoid visual-design distraction.

18.4 Laws of UX

Evidence-based principles from psychology and human behaviour that support design decisions.

- **Aesthetic-Usability Effect:** attractive design is perceived as more usable; creates a positive halo but can mask minor usability issues; aesthetics complement, not replace, real usability.
- **Choice Overload:** too many options overwhelm users and reduce decision quality; satisfaction decreases as option count increases (paradox of choice); limit options to what is necessary.
- **Chunking:** breaking information into smaller, meaningful groups improves comprehension and reduces cognitive load; effective chunk size is 3–5 items; use visual separation (spacing, borders).
- **Doherty Threshold:** system response under 400 ms keeps users in flow; delays >1 s interrupt thought; use loading indicators for unavoidable delays.
- **Fitt's Law:** time to acquire a target depends on distance and target size; larger/closer targets are faster; edge targets are effectively infinite in size; place important actions (Save, Submit) near the cursor.
- **Flow:** complete immersion in a task (clear goals, immediate feedback, balanced challenge); interruptions (popups, slow loads) break flow; flow leads to high satisfaction and repeat usage.
- **Jakob's Law:** users prefer your site to work like sites they already know; familiar patterns reduce learning curve; innovate in functionality, not in basic interaction patterns.
- **Law of Proximity:** objects near each other are perceived as related; proximity overrides colour and shape for grouping; white space is a powerful grouping tool.
- **Law of Similarity:** similar elements (colour, shape, size) are perceived as related; consistent styling signals functional relationships; different action types need distinct styling.

18.5 Tools, Systems, and Collaboration

- **UI Designer vs. UX Designer:** UX (how it works and feels — research, flows, wireframes, testing) comes first; UI (how it looks — colours, typography, visual hierarchy) comes second; both roles collaborate closely.
- **Design system:** a collection of reusable components, patterns, and standards (UI components, design tokens, usage guidelines); ensures consistency across products/teams/ platforms; speeds up development by eliminating repeated decisions. Examples: Google Material Design, IBM Carbon, Atlassian Design System, Shopify Polaris.
- **UI kits:** pre-designed interface elements (buttons, inputs, cards, modals, icons, nav bars); often the visual implementation of a design system's components; examples: Figma UI kits, Bootstrap, Ant Design, Chakra UI, Tailwind UI.
- **Figma:** cloud-based, real-time collaborative design tool; supports wireframing, high-fi mockups, interactive prototypes, and developer handoff; key features: auto layout, components, variants, plugins, version history; the current industry standard.

V Engineering Practices *(Course Extras)*

19 Introduction to DevOps

19.1 What is DevOps?

- A cultural and technical movement that bridges development (Dev) and operations (Ops) teams.
- Emphasises collaboration, automation, and shared responsibility throughout the software lifecycle.
- Goal: deliver software faster, more reliably, and with fewer errors.
- Key practices: continuous integration, continuous delivery, infrastructure as code, monitoring, and logging.
- DevOps is *not* a tool or a job title — it is a mindset and a set of practices.

19.2 The Problems Before DevOps

- Development and operations worked in separate silos with conflicting goals: Dev wanted to push changes fast; Ops wanted stability.
- Long release cycles with integration happening at the end caused "integration hell."
- Deployments were manual, risky, and infrequent (weeks or months).
- Bugs found late in the cycle were expensive and time-consuming to fix.

19.3 How DevOps Works

- Breaks down silos by uniting development, operations, and QA; shared responsibility.
- Automation replaces manual handoffs and repetitive tasks.
- Feedback loops are shortened: issues caught earlier, fixed faster.
- Outcome: faster delivery, higher quality, less friction.

19.4 CI/CD

- **Continuous Integration (CI):** developers merge code into a shared repository multiple times per day; requires automated builds and tests to catch integration issues immediately.
- **Continuous Delivery (CD):** software is always in a deployable state after the CI pipeline passes; automates the release process so any passing build can be deployed with one click.
- Together, CI/CD enables faster feedback, lower risk, and frequent reliable deployments.

19.5 DevOps, Agile, and System Design

- Agile produces potentially shippable code; DevOps ensures it is *actually* deployable at the end of every sprint by automating the "last mile" from commit to production.
- **Agile + DevOps** = fast development AND fast, reliable deployment.
- DevOps shapes system design: **design for deployability** (zero-downtime blue-green/ canary), **observability** (logs/metrics/traces built in from day one), **automation** (infra provisioned as code), **failure** (graceful rollback), and **testability** (each component independently testable).

19.6 The CI/CD Pipeline

- **Source:** developer pushes code to version control (Git).
- **Build:** compile, resolve dependencies, create artifacts.
- **Test:** unit tests, integration tests, static analysis.
- **Deploy to staging:** production-like environment for further validation.
- **Release to production:** after approval — automatically or manually.

19.7 Environments, Tools, and Culture

- **Environments** are isolated system instances for different lifecycle stages: development (local coding/testing), staging (pre-production validation), production (live users).
- **Your DevOps tool stack:** version control (Git/GitHub/GitLab), CI/CD (Jenkins/ GitHub Actions/GitLab CI), containerisation and orchestration (Docker, Kubernetes), IaC (Terraform, Ansible), monitoring (Prometheus, Grafana).
- **Infrastructure as Code (IaC):** manage and provision infrastructure through machine-readable definition files; servers, networks, databases, and load balancers defined as code in version control; repeatable, testable, and versioned like application code.
- **A pipeline alone does not mean DevOps:** true DevOps requires the culture — shared responsibility, collaboration, and continuous improvement; without the culture it is just automation.

20 Docker

20.1 Why Containers

- Solves "it works on my machine": the app and *all* its dependencies travel together and run consistently everywhere; deployment is faster and more reliable.
- **Containers vs. VMs:** VMs bundle a full guest OS; containers *share the host kernel*, so they are smaller, start faster, and pack more per host. VMs give stronger isolation at higher overhead.

20.2 Ecosystem & Concepts

- Workflow: **Dockerfile** → **Image** → **Registry** → **Container**.
- An **image** is a read-only template; a **container** is a running instance of an image with its own processes/network/filesystem changes.
- **Lifecycle:** `build` an image → `run` a container → stop/restart/pause → remove and recreate from the same image.
- **Under the hood:** Linux *namespaces* (isolation), *cgroups* (resource limits), layered images, and a background *daemon*. On macOS/Windows, Docker Desktop runs a Linux VM / WSL 2 so the same Linux container runs everywhere.

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Listing 28: Single-stage Dockerfile (from the slides)

```
# build stage: has compilers/build tools
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN go build -o /bin/app
```

```
# final stage: only the artifact, smaller + smaller attack surface
FROM gcr.io/distroless/static
COPY --from=build /bin/app /app
ENTRYPOINT ["/app"]
```

Listing 29: Multi-stage build keeps the final image small (added)

20.3 Dockerfile Details

- **Layers:** each instruction is an immutable, cacheable layer; put frequently changing steps near the end to maximise cache reuse.
- **CMD vs. ENTRYPOINT:** `ENTRYPOINT` is the always-run executable; `CMD` gives default args/command. `docker run` args override `CMD` but are *appended* to `ENTRYPOINT`. Common pattern: `ENTRYPOINT` = app, `CMD` = default options. *For a multi-directive startup, use an entrypoint script:* `ENTRYPOINT ["./entrypoint.sh"]`.
- **ADD vs. COPY:** prefer `COPY`; use `ADD` only for its extras (auto-extract local archives, fetch URLs — rarely recommended).
- **EXPOSE vs. port mapping:** `EXPOSE` only documents a port; `-p 8080:80` (host:container) actually publishes it.

```
docker build -t myapp:1.0 .        # image from Dockerfile
docker run -d -p 8080:80 myapp     # start container, publish port
docker ps                          # running containers
docker stop <id>                   # graceful stop
docker images                      # local images
```

Listing 30: Common Docker commands (added)

20.4 Compose, Volumes, Networks

- **Docker Compose** declares multi-container apps in `compose.yaml`; `docker compose up` starts the whole stack and a dedicated network.
- **Volumes** give persistent storage independent of a container's life; shareable across containers; mount read-only with `:ro` when you only need to read.
- **Networks:** `bridge` (default, same host), `host` (no isolation), `none` (no networking), `overlay` (multi-host clusters), `macvlan` (own MAC/IP). Containers on the same network reach each other by name.
- **Registries:** public Docker Hub (repos + tags like `latest`/`v1.0`); private (Artifactory, Nexus) for access control, scanning, compliance.
- **Swarm** is Docker's native orchestrator (replicas, load balancing, failover) but is largely superseded by Kubernetes. **Podman** is a daemonless, rootless, Docker-compatible alternative.

```
services:
  web:
    build: .
    ports: ["8080:80"]
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - dbdata:/var/lib/postgresql/data        # persistent
      - ./seed.sql:/docker-entrypoint-initdb.d/seed.sql:ro  # read-only
volumes:
  dbdata:
```

Listing 31: compose.yaml: web + db with a volume and read-only mount (added)

21 DevOps & CI/CD Tools

21.1 Pipelines

- CI/CD tools automate build/test/deploy, reduce human error, give fast feedback, and create a shared, consistent workflow.
- A **pipeline** is an automated sequence of stages (integrate → build → test → security → deploy); each stage validates before the next; fail early.
- **Designing a pipeline:** identify build/test/deploy stages, add quality gates (unit tests, static analysis, security scans), order stages to fail fast, configure dev/staging/prod deploys, then monitor and optimise.
- **Infrastructure as Code (IaC):** provision via version-controlled config (Terraform, Ansible) for repeatable, consistent environments.

21.2 The Tools

- **Jenkins:** open-source automation server; huge plugin ecosystem; pipelines as code in a `Jenkinsfile`.
- **GitHub Actions:** CI/CD built into GitHub; workflows in `.github/workflows/*.yml`; large action marketplace; no extra servers.
- **GitLab CI/CD:** built-in; pipelines in `.gitlab-ci.yml`; jobs run on Runners.
- **CircleCI:** cloud CI/CD; YAML config; parallelism and caching.
- **Terraform** (HashiCorp): declarative IaC; multi-provider (AWS/Azure/GCP/Kubernetes); shows an execution plan before applying.
- **Artifact repositories** (Artifactory, Nexus, Docker Hub, GHCR): store versioned build outputs so the *same* build deploys to every environment.
- **Kubernetes (K8s):** container orchestration — scheduling, load balancing, service discovery, rolling updates, self-healing, scaling.
- **AI in CI/CD:** code generation/review, predicting build failures/anomalies, test generation/prioritisation, and intelligent log/metric analysis for faster incident response.

21.3 Real-World Flow

Push to GitHub triggers Actions → build + test + scan → Docker builds an image pushed to a registry → Terraform provisions/updates infra → deploy to production → monitor.

```yaml
name: ci
on: [push]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest --cov=app
      - run: docker build -t ghcr.io/org/app:${{ github.sha }} .
```

Listing 32: GitHub Actions workflow (added)

```
pipeline {
  agent any
  stages {
    stage('Build') { steps { sh 'make build' } }
    stage('Test')  { steps { sh 'make test'  } }
    stage('Deploy'){ when { branch 'main' }
                     steps { sh 'make deploy' } }
  }
}
```

Listing 33: Declarative Jenkinsfile (added)

22 Microservices in Practice

22.1 From Monolith to Services

- Monolith pains: huge codebases, one failure can sink everything, must scale the whole system, team conflicts, slow/risky deploys.
- Cloud computing + pioneers (Netflix, Amazon) made small independent services practical: each owns a business capability, communicates via lightweight APIs/messaging, and deploys and scales independently.

22.2 Designing Services with DDD

- Organise around business capabilities, not technical layers; clear single responsibility; minimal cross-service dependencies; stable APIs; "small enough to manage, large enough to be meaningful."
- **Domain-Driven Design:** model around real domains; split into **bounded contexts** (often one per microservice).
- **Ubiquitous language:** one shared vocabulary across talk, docs, and code to prevent misunderstanding.
- **Teams** own a service's full lifecycle; autonomy enables faster delivery and free technology choice.

22.3 Communication

- Synchronous (REST, gRPC) or asynchronous (message brokers); keep services loosely coupled; handle timeouts/retries/errors.
- **gRPC:** high-performance RPC using Protocol Buffers; binary serialisation (faster than JSON); multi-language code generation; great for low-latency calls.
- **Service discovery:** register/find instances dynamically (Consul, Eureka, K8s DNS).
- **API gateway:** single client entry point for auth, routing, rate limiting, aggregation (Kong, NGINX, Spring Cloud Gateway).
- **Workflows** span services via orchestration or choreography; design for failures, retries, and consistency.

```
syntax = "proto3";
service OrderService {
  rpc PlaceOrder (OrderRequest) returns (OrderReply);
}
message OrderRequest { string item = 1; int32 qty = 2; }
message OrderReply   { string order_id = 1; bool ok = 2; }
```

Listing 34: gRPC service contract (.proto) (added)

22.4 Building, Releasing, Testing, Operating

- Independent deployable units behind well-defined, *versioned* APIs; package with containers and CI/CD.
- **Release strategies:** blue-green and canary reduce risk; keep rollback/monitoring.
- **Testing layers:** unit, integration, *contract* (API compatibility between services), end-to-end.
- **Security:** authn/authz, TLS everywhere, secure storage/transit, least privilege, regular security testing.
- **Observability:** metrics (health/perf), centralised logging, distributed tracing across services.
- **Challenges:** operational/architectural complexity, unreliable network calls, distributed data, hard debugging — needs strong automation/DevOps.
- *Do you need them?* Often a monolith is more efficient for small projects; benefits grow with system/team size. Decide by business need, not trend.

23 More Software Development Methodologies

- **Disciplined Agile Delivery (DAD)** — Ambler & Lines, 2012: a process-*decision* framework covering the full delivery lifecycle, blending Scrum/XP/Kanban/Lean/Agile Modeling, with flexibility plus governance. Phases: *Inception*, *Construction*, *Transition*.
- **Dynamic Systems Development Method (DSDM):** agile delivery with strong governance; early/frequent business value; active user involvement. Roles: Business Sponsor, Business Visionary, Technical Coordinator, Team Leader, Solution Developers/Testers. Phases: Feasibility → Foundations → Evolutionary Development → Deployment → Post-Project.
- **Rational Unified Process (RUP):** iterative (Rational/IBM); disciplined PM, documentation, continuous risk reduction. Phases: Inception → Elaboration → Construction → Transition, each with multiple iterations.
- **Lean Software Development:** Toyota-inspired; maximise customer value, eliminate waste, decide from learning/feedback, deliver in small increments, improve continuously.
- **There is no single "correct" methodology:** pick and adapt practices to goals, constraints, and stakeholders; the best one consistently delivers value, quality, and satisfaction.

24 Software Documentation

- **Problem it solves:** early projects relied on individual knowledge; undocumented decisions made systems hard to understand, maintain, and grow.
- **What to document:** requirements/specs, architecture & design decisions, code organisation/key components, install/deploy/config, user guides & maintenance.
- **What *not* to document:** obvious self-explanatory code, duplicated info, temporary details, outdated/inaccurate info, excessive detail.
- **Types:** user, developer, system, API, process documentation.
- **Good docs** are clear, accurate, appropriately detailed, well organised, and kept up to date.
- **Best practices:** write for the audience; concise language; examples/diagrams; keep in sync with the software; consistent organisation.
- **Tools:** Markdown, Doxygen/Javadoc (generate from source), Sphinx, MkDocs, Swagger (APIs), wikis, version control.
- **Reading docs:** identify purpose/audience; scan the table of contents; focus on task-relevant sections; study examples/diagrams; follow references.
- **References vs. guides:** references give precise technical detail (functions, classes, APIs); guides teach tasks step-by-step. Good documentation provides both.
- **Key takeaway:** documentation is part of development, differs by stakeholder, must stay accurate/current, reduces maintenance/risk, and evolves with the system.

```python
def transfer(amount: float, src: str, dst: str) -> bool:
    """Move funds between accounts.

    Args:
        amount: positive amount to transfer.
        src: source account id.
        dst: destination account id.
    Returns:
        True if the transfer succeeded.
    Raises:
        ValueError: if amount <= 0.
    """
```

Listing 35: Docstring that Sphinx/IDEs can render (added)

25 Software Testing

- Evaluating a system to detect defects; ensures requirements are met; improves quality, reliability, and satisfaction; manual or automated; reduces post-deployment failure risk.
- **Throughout the SDLC:** starts at requirements (catch issues early), developer testing during implementation, dedicated validation phases, and continuous testing during maintenance so changes don't introduce regressions.

25.1 Methods & Levels

- **Ad-hoc:** informal exploration, no scripted cases; best with strong domain knowledge.
- **Manual:** act as an end user; great for usability/UX/visuals.
- **Automated:** tools/scripts; ideal for repetitive tests and CI/CD; upfront cost, long-term savings.
- **Unit:** individual functions/classes in isolation; written by developers; JUnit, NUnit, pytest.
- **Integration:** interfaces and data exchange between modules.
- **System:** the complete integrated app against all requirements, from a user view.
- **Smoke:** quick check that critical functionality works after a build.
- **Regression:** ensure changes haven't broken existing features; usually automated.

25.2 Coverage, Edge Cases, Critical Paths

- **Coverage** (statements/branches/functions) shows what tests exercise; higher is better but 100% doesn't guarantee bug-free; use it to find untested areas.
- **Edge cases:** boundary/min/max/unusual inputs — rare but defect-prone.
- **Critical paths:** the most-used, highest-impact workflows (auth, payments, data persistence) — test these first.

25.3 Mocking & Test Design

- **Mocking / test doubles** replace real dependencies (DB, APIs, filesystem) with configurable simulations for isolated, fast, controllable tests.
- *Clean Architecture helps mocking* because the dependency rule and interfaces in the inner layers let you substitute outer-layer implementations (DB/HTTP) with test doubles — the use cases depend on abstractions, not concrete tools.
- **Design tests** from requirements; cover normal, edge, and invalid inputs; prioritise critical workflows; keep tests independent/repeatable/readable; use coverage to find gaps.

```python
from unittest.mock import Mock

def notify(user_id, gateway):
    return gateway.send(user_id, "hi")

def test_notify_calls_gateway():
    gw = Mock()                        # test double
    gw.send.return_value = True
    assert notify(42, gw) is True
    gw.send.assert_called_once_with(42, "hi")
```

Listing 36: Unit test with a mock (pytest) (added)

25.4 Higher-Level & Specialised Testing

- **End-to-End (E2E):** complete user workflows in a production-like environment; valuable but slow/expensive to maintain.
- **In Continuous Delivery:** automated unit/integration/E2E run at multiple pipeline stages; fast feedback; results gate promotion; enables frequent quality releases.
- **Black-box vs. white-box:** behaviour from inputs/outputs vs. internal logic/paths/coverage — complementary.
- **Shadow testing:** run a new system alongside production on real inputs (no user impact) and compare outputs to find discrepancies before release.
- **User Acceptance Testing (UAT):** business stakeholders validate real workflows; the final gate before deployment.
- **Performance testing:** behaviour under load — response time, throughput, resource use, bottlenecks. **Security testing:** vulnerabilities — unauthorised access, data leaks, injection. Together they ensure reliability and resilience.

VI Epilogue

26 The Epilogue: Looking Back & Forward

26.1 What This Course Covered

- Planning, Agile, Scrum, and requirements gathering.
- Software architecture characteristics and patterns (MVC, Clean, Microservices).
- UML: structural diagrams (Class, Object, Component, Deployment) and behavioural diagrams (Use Case, Sequence, Activity, State).
- System design: DNS, load balancers, caching, message queues, stateless architecture, CDNs, and object storage.
- UI/UX: laws of UX, wireframes, prototypes, design systems, and Figma.
- DevOps: CI/CD, pipelines, infrastructure as code, Docker, Kubernetes.

26.2 Career Paths

- **Product Owner:** defines requirements, manages the backlog, and prioritises features for business value.
- **Architecture Owner:** designs the high-level structure, chooses patterns, and ensures technical quality.
- **DevOps Engineer:** builds CI/CD pipelines, manages infrastructure, and automates deployments.
- **UI/UX Designer:** researches users, creates wireframes and prototypes, and crafts intuitive interfaces.
- **Software Engineer (SWE):** writes code, implements features, fixes bugs, and makes the design real.

26.3 AI and the Software Engineer

- AI will *augment* SWEs, not replace them; it handles boilerplate, suggestions, and simple tasks.
- Complex system design, architecture decisions, and trade-off reasoning still require human judgement.
- Understanding business requirements and translating them into technical solutions is a human skill; so is debugging subtle issues, performance tuning, and maintaining legacy systems.
- SWEs who use AI become more productive; those who ignore it may fall behind.
- Use AI for boilerplate, refactoring, explaining concepts, and reviewing designs — then focus your own energy on creativity, architecture, and problem-solving.

26.4 Soft Skills and Mindset

- **Communication:** explaining technical decisions to non-technical stakeholders is a superpower.
- **Trade-offs:** every decision involves trade-offs; there is no perfect solution.
- **Documentation:** future you and your teammates will thank you for clear, up-to-date docs.
- **Continuous learning:** technology changes fast; fundamentals (patterns, principles) stay relevant.
- **Fail fast, learn faster:** mistakes in design, code, or deployment are opportunities, not failures.

26.5 Practical Next Steps

- Build something small end-to-end: from requirements to UI to backend to deployment.
- Contribute to open source to see how real systems are designed and reviewed.
- Practice system design with friends or on a whiteboard.
- Learn one tool deeply from each category (Git, Docker, Figma, etc.) instead of skimming all.
- Find a mentor or community to review your designs and code.
- You don't need to master everything at once — start with what you use daily; every expert was once a beginner who refused to give up.

VII Cheat Sheet

A condensed, exam-oriented reference distilled from the whole course.

27 Methodology Selection "Pick Your Poison"

Usefulness for	Water-fall	Parallel	Phased	Proto-typing	Throwaway	XP
Unclear user requirements	Poor	Poor	Good	Great	Great	Great
Unfamiliar technology	Poor	Poor	Good	Poor	Great	Poor
Complex systems	Good	Good	Good	Poor	Great	Poor
Reliable systems	Good	Good	Good	Poor	Great	Good
Short time schedule	Poor	Good	Great	Great	Good	Great
Schedule visibility	Poor	Poor	Great	Great	Good	Good

28 Roles, Requirements & Quick Mnemonics

- **Dev-process roles:** owner (funds) · user (uses) · analyst (what) · designer (how) · builder (constructs) · PM (controls) · ESP (external).
- **Scrum roles:** Product Owner (value, backlog) · Scrum Master (process, impediments) · Dev Team (build, self-organise, 5–9).
- **Scrum artifacts:** Product Backlog · Sprint Backlog · Increment.
- **Scrum events:** Sprint · Sprint Planning · Daily Stand-up ($\leq$15 min) · Review · Retrospective · Backlog Refinement.
- **DoR** (ready before sprint) vs. **DoD** (done = shippable).
- **3 Cs:** Card · Conversation · Confirmation.
- **Story sizes:** Epic (months) > Theme (weeks) > Sprintable story (days) > Task (hours).

- **INVEST:** Independent, Negotiable, Valuable, Estimatable, Small, Testable.
- **DEEP backlog:** Detailed appropriately, Emergent, Estimated, Prioritized.
- **Retro questions:** continue / stop / start.
- **Burndown** = remaining work down to 0; **Burnup** = completed work up to target.

29 Feasibility & Economics

- **Feasibility types:** Operational, Technical, Economic, Schedule, Legal, Cultural.
- **Matrices:** Candidate Systems (compare) · Feasibility Analysis (rank, weighted).

$$PV = \frac{FV}{(1+i)^n} \quad \text{ROI} = \frac{B-C}{C} \cdot 100\% \quad \text{NPV} = \sum_{t=0}^{n} \frac{B_t - C_t}{(1+i)^t} \text{ (accept if } > 0)$$

$$\text{People} = \frac{\text{person-months}}{\text{months}} \quad \text{Comm. lines} = \frac{n(n-1)}{2} \quad \text{Sprints} = \frac{\text{size}}{\text{velocity}}$$

30 UML & Modeling

- **Structural diagrams:** class, package, deployment.
- **Behavioral diagrams:** use case, sequence, activity, state machine.
- **Use-case relations:** association · ⟨⟨uses/includes⟩⟩ · ⟨⟨extends⟩⟩ · ⟨⟨depends on⟩⟩ · inheritance.
- **State machine:** state, transition, event, guard, action, initial/final, composite, superstate, concurrent regions.
- **Process models:** DFD, BPM/BPMN (events, activities, gateways, sequence flows, pools/lanes). **Data model:** ERD. **Object model:** class/sequence/collaboration.

31 Pattern Catalogue

Category	Patterns
Architectural	MVC, MVVM, Layered (n-tier), Clean, Microkernel, Microservices
Creational	Abstract Factory, Builder, Factory Method, Prototype, Singleton
Structural	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
Behavioral	Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

One-line intents (most-tested)

- **Singleton** — one instance, global access.
- **Factory Method** — subclass decides the concrete class.
- **Abstract Factory** — create whole families of related objects.
- **Builder** — step-by-step construction of complex objects.
- **Adapter** — make an incompatible interface fit.
- **Decorator** — add responsibilities by wrapping.
- **Facade** — one simple interface over a complex subsystem.
- **Proxy** — controlled/lazy access via the same interface.
- **Composite** — treat part and whole uniformly (trees).

- **Observer** — one-to-many auto-notification (events).
- **Strategy** — interchangeable algorithms chosen at runtime.
- **Command** — request as object (queue/log/undo).
- **State** — behaviour changes with state (FSM).
- **Template Method** — fixed skeleton, customisable steps.
- **Chain of Responsibility** — pass a request along handlers.

32 DevOps, Docker & Testing Quick Reference

- **Docker flow:** Dockerfile → Image → Registry → Container.
- **Image** = read-only template; **Container** = running instance.
- **CMD** = default args (overridable); **ENTRYPOINT** = always-run executable.
- **COPY** preferred over **ADD**; **EXPOSE** documents, `-p` publishes.
- **Networks:** bridge / host / none / overlay / macvlan.
- **Pipeline stages:** integrate → build → test → security → deploy (fail early).
- **IaC:** Terraform, Ansible. **Orchestration:** Kubernetes.
- **Test levels:** unit → integration → system → E2E; plus smoke, regression, contract, shadow, UAT, performance, security.
- **Black-box** (behaviour) vs. **white-box** (internal logic/coverage).
- **Release strategies:** blue-green, canary.

33 Microservices & Architecture Decisions

- **DDD:** bounded contexts (≈ one per service) + ubiquitous language.
- **Comms:** REST/gRPC (sync) or message brokers (async); loose coupling; retries/timeouts.
- **Infra:** API gateway, service discovery, observability (metrics, logs, tracing).
- **Rule of thumb:** start with a monolith; split to microservices when team/system size and differing scaling needs justify the operational cost.

34 System Design Quick Reference

- **Client–Server:** presentation/interaction (client) vs. data/logic/persistence (server).
- **DNS:** domain names → IPs; enables load distribution, redundancy, service mobility.
- **SQL vs. NoSQL:** SQL = ACID, structured, joins; NoSQL = flexible schema, horizontal scaling, high write throughput.
- **Scaling:** vertical (add power to one server, limited); horizontal (add servers, complex but near-infinite). SQL scales vertically; NoSQL scales horizontally.
- **Load balancer:** distributes traffic; hides failures. Algorithms: round robin, least connections, IP hash.
- **DB replication:** master (writes) → slaves (reads); slave promoted on master failure.
- **Cache:** serve from memory; strategies: read-through, write-through, write-around, write-back. Set TTL; handle invalidation. Watch for stampede.
- **CDN:** edge servers geographically near users; caches static assets; reduces latency and origin load.

- **Stateless architecture:** no server-side session; any server can handle any request; session in DB/Redis/JWT. Enables horizontal scaling.
- **GeoDNS:** returns different IPs by user location; combines with CDNs/load balancers.
- **Message queue:** async decoupling; producers write, consumers read at own pace. Examples: RabbitMQ, Kafka, SQS.
- **Observability stack:** ELK (logs), Prometheus + Grafana (metrics), distributed tracing.
- **API gateway:** single entry point; routing, auth, rate limiting, aggregation (NGINX).
- **Object storage:** unstructured data (images, video, backups) as objects with metadata and unique IDs; S3-compatible (MinIO).
- **Arch characteristics:** Availability, Reliability, Testability, Scalability, Security, Agility, Fault Tolerance, Elasticity, Recoverability, Performance, Deployability, Learnability.

35 UI/UX Laws Quick Reference

- **Aesthetic-Usability Effect:** pretty = perceived as more usable (but can mask flaws).
- **Choice Overload:** fewer options = better decisions; limit and group choices.
- **Chunking:** group related items in 3–5 chunks; use spacing/borders.
- **Doherty Threshold:** $<$400 ms response keeps users in flow.
- **Fitt's Law:** bigger + closer targets are faster to reach; edges are effectively infinite.
- **Flow:** clear goals + immediate feedback + balanced challenge = deep engagement.
- **Jakob's Law:** follow familiar conventions; innovate in functionality, not in interaction patterns.
- **Law of Proximity:** nearby = related; use white space deliberately.
- **Law of Similarity:** similar look = similar function; distinguish primary vs. secondary actions.
- **UX process:** Requirements $\rightarrow$ Design $\rightarrow$ Verification $\rightarrow$ Implementation $\rightarrow$ Maintenance.
- **Fidelity:** Sketch (lo-fi, diverge) $\rightarrow$ Wireframe (structure) $\rightarrow$ Prototype (interactive test).

36 DevOps Quick Reference

- **CI** = merge often + auto build/test. **CD** = always deployable + one-click release.
- **Pipeline stages:** Source $\rightarrow$ Build $\rightarrow$ Test $\rightarrow$ Staging $\rightarrow$ Production.
- **Environments:** Development (local) $\rightarrow$ Staging (production-like) $\rightarrow$ Production (live).
- **IaC:** infra defined as version-controlled code (Terraform, Ansible, CloudFormation).
- **Pipeline $\neq$ DevOps:** pipeline is a tool; DevOps is shared responsibility + culture.
- **Design for DevOps:** deployability, observability, automation, failure recovery, testability.
- **Agile + DevOps:** Agile = working software every sprint; DevOps = deployed every sprint.